CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

CI/CD helps reduce the time gap between code modifications and their impacts on the running application services. Although it enables rapid delivery of applications, CI/CD also enlarges the number of artifacts which have to be investigated in case of any problems. This includes output from CI/CD pipelines describing building and testing processes, artifacts produced during the release with metadata of the version released, Git history with latest modifications, and symptoms in logs/health checks. Artifacts are usually accessed via different tools. The troubleshooter needs to pick up right events, correlate the timestamps, and separate a coincidental change from a possible cause.

An incident starts with a symptom, not a known cause. A faulty health test may be an indicator of an exception that occurred during application operation, an unavailability of some dependency, a failed deployment, or an alteration in configuration. One log statement may show a failure but will not reveal the trigger. The issue, in other words, is not just finding data; the problem lies in selecting the evidence, arranging it chronologically, choosing among alternative causes, and deciding if enough is known.

RecoverIT was developed to support this investigation sequence. It treats source systems as read-only inputs, represents their observations through typed contracts, forms a bounded incident context, and presents ranked hypotheses with traceable evidence. Language-model output is advisory: deterministic rules validate references, scoring and final recommendations. Such a division aims at making the output inspectable to an operator, and keeping the output from being transformed into a command by the free-response model.

1.2 PROBLEM STATEMENT

The current approach to incident response frequently relies on engineers correlating CI/CD and production evidence manually in a tight timeframe. An effective assistant needs to collect pertinent evidence, track provenance, construct a timeline, assess multiple causes of failure, and acknowledge uncertainty when stopping for lack of evidence. The system must also ensure that any suggested next action remains

reviewable by a human. RecoverIT addresses this bounded investigation and recommendation problem; it does not currently perform autonomous recovery.

1.3 OBJECTIVES

Objective	Implementation evidence
Unify incident inputs	Typed incident and evidence contracts; configurable source registry
Preserve source provenance	Normalization, source identifiers, timestamps and evidence records
Construct context	Timeline, relationship and context services
Rank causes	Hypothesis service, deterministic ranking engine and stopping rules
Control operational risk	Read-only adapters and remediation policy validation
Provide usable output	CLI report, HTTP API and web interface

Table 1.1 Project objectives and evidence

The first objective is to make the intake and source boundary explicit. A canonical incident seed captures the affected service and time context. Registered adapters advertise capabilities so the graph can request only supported data. Collection failures are represented as source outcomes rather than silently turned into evidence or allowed to terminate the whole investigation.

The second goal is the preservation of evidence identity and quality. The data gets normalized into a common form while keeping its own identifiers and timestamp from the source. The redaction and the quality check take place before context becomes available for reasoning. Duplicate observations get consolidated in order to avoid duplication as corroboration.

The third objective is to produce a bounded, inspectable diagnosis. The investigation graph can ask for further information, collect it, update context and compare hypotheses. A deterministic ranking stage evaluates independent sources, symptom coverage, temporal and change consistency, specificity, predictions, contradictions and missing causal evidence. Stopping rules allow completed or inconclusive outcomes.

The fourth objective is to deliver responsible operator guidance. A remediation plan is a descriptive recommendation, with prerequisites, verification and reversal guidance. Policy checks prevent unsupported changes. The current system has no write-capable remediation executor and makes no claim of post-action service recovery.

1.4 SCOPE AND CONSTRAINTS

The implemented scope covers a configured live runtime with real read-only adapters, a provider-neutral reasoning layer, a LangGraph investigation workflow, checkpoint persistence, ranked hypotheses, a report representation and optional recommendation-only remediation. This solution will be useful for the CI/CD process and any incidents for which there are sources that have relevant records. It will depend on the amount of data available from these sources.

This particular solution does not provide for any production accuracy, mean time to resolution reduction or self-healing capabilities. While the earlier designs did propose some self-healing goals, this particular solution purposely fails to implement them before write operations. Rollbacks, restarting services, scaling and any validation of actions taken will be done in later versions.

The implementation is constrained by provider availability and credentials, adapter coverage, finite investigation budgets and the reliability of external records. The model may generate an inaccurate candidate explanation; consequently, schema validation, provenance checks, deterministic ranking and abstention are part of the design rather than optional presentation features.

1.5 EXPECTED OUTCOMES AND REPORT ORGANISATION

The expected result of Project I is an operational investigation application with a repository that proves the path from the canonical event to the hypothesis linked with evidence and the safe advice. Chapter 2 provides a review of the literature that influenced the design. Chapter 3 describes the architecture and approaches used. The results of implementation and direct observation of verification are stated in Chapter 4. Conclusions are drawn in Chapter 5. The necessary supporting interfaces, contracts, tests and contributions are provided in the appendices.

CHAPTER 2

LITERATURE REVIEW

2.1 REVIEW APPROACH

The provided literature survey consists of sixteen papers about autonomous SRE, language-model agents, RCA of microservices, CI/CD failure diagnoses, observability, and self-healing. While maintaining the scope of the literature survey, this chapter will evaluate every paper based on the methodology, evidence needs, limitations, and relevance to RecoverIT. The numbers quoted in this section relate to the sources cited and do not correspond to this project at all. The review was conducted before the definition of the RecoverIT scope became complete; references to automated repair serve as motivation only.

2.2 MULTI-AGENT SELF-HEALING

Manohar, Shukla & Abi (2025) discuss an autonomous self-healing model for Site Reliability Engineering that incorporates multi-agent architecture for automation of incident management process in cloud-native ecosystems. Unlike conventional methods of automation that focus on only generating alerts in the incident management process, the proposed system is built to work in the entire process lifecycle. The authors implement the framework using CrewAI and Large Language Models including DeepSeek-R1, GPT-4o, and Gemini 2.5 Flash. The framework is organized into several layers. A Kubernetes infrastructure layer hosts both the target application and the autonomous agent system.

An application layer contains the monitored application and a monitoring agent integrated with Middleware.io, which continuously observes application performance and system health. When an anomaly is detected, an alert starts the self-healing workflow. The central Autonomous SRE Agent layer contains the specialized agents that collaborate on the incident, while an external dependency layer connects the system with services such as Middleware.io, Jira Service Management, GitHub, and Langfuse. The multi-agent system contains four main specialized roles.Incident Resolution Manager will be responsible for orchestration and will manage the incident life cycle and will receive the alert and prioritize and delegate tasks to other agents.

Jira Manager will be responsible for incident ticket management, which includes creation, modification of the findings, change ticket status and incident life cycle management. Root Cause Analyzer will perform the root cause analysis and correlation of application logs of Middleware.io, Kubernetes cluster state, and source code on GitHub. Code Fix Generator will take the output of RCA and generate code changes and create new git branch and submit pull request. The entire workflow is implemented in the paper as a sequential incident resolution process. An incident starts when Jira ticket is created.

The Incident Resolution Manager selects high-priority incidents and delegates investigation to the appropriate agents. The Root Cause Analyzer collects information from the available operational sources and produces an RCA report, which is added to the Jira ticket. After RCA is completed, the Code Fix Generator uses the report to implement the required remediation and creates a GitHub pull request. A human engineer then reviews the generated pull request before it can be merged. If the incident is too complex for autonomous resolution, it is returned for manual intervention.

After approval and merging, the incident ticket is transitioned to completion. The implementation was developed using Python 3.12 and CrewAI. The agents have been designed using prompt engineering where their duties, objectives, and workflow have been mentioned. The Root Cause Analyzer has been tasked with correlating the data from Jira ticket details, Kubernetes State, application logs, and source code to find the piece of code or configuration responsible for the issue. The Code Fix Generator has been tasked with carrying out the fix plan.

Custom tools based on REST APIs were developed for Jira, Kubernetes, and GitHub interactions, including a Jira MCP Tool, Kubernetes analysis tools, and GitHub MCP Tools. The complete application was containerized and deployed on Kubernetes. An important part of the implementation is its emphasis on auditability. Langfuse is integrated to record agent interactions and LLM calls throughout the workflow. It helps monitor the activities and decisions made within the multi-agent system. This helps inspect the process of resolving the incident after it has been executed.

The human-in-the-loop phase gives another layer of control as the generated code is reviewed before merging into the master branch. The evaluation of the framework is done in an environment that imitates a real-life production SRE environment. A faulty

web application is deployed on Kubernetes and engineered to generate operational failures such as internal server errors, database connection timeouts, and pod crashes. The environment is integrated with Middleware.io, Jira Service Management, GitHub, and Langfuse so that the complete incident lifecycle can be evaluated. Gemini 2.5 Pro is used as an independent judge to assess the quality and completeness of tasks performed by the agents.

The evaluation reports 71% correctness in agent task delegation, 68% precision for identifying actionable root causes, and up to 96% accuracy for simpler RCA tasks. The Code Fix Generator successfully produced valid pull requests for 73% of the tested cases. The JIRA Manager achieved 97% compliance with the expected ticket-state workflow, while the average end-to-end execution time was 27.48 minutes. Langfuse recorded 100% of the workflow executions, providing complete telemetry of the agent interactions. The results demonstrate that the Root Cause Analyzer can correlate data from various sources to create incident analyses, but its effectiveness diminishes as the complexity of the problems increases.

The Code Fix Generator shows that the system is capable of taking action and modifying code after diagnosing it instead of stopping at an RCA analysis. The authors therefore demonstrate an end-to-end approach in which LLM agents can coordinate operational investigation and software remediation while retaining human approval for the final change. The paper also identifies future extensions to the framework. The authors propose a long-term memory graph so that agents can learn from historical incidents and reuse proven resolution strategies, together with a searchable knowledge base of previous problems. They also specifically identify deeper integration with CI/CD pipelines and correlation of incidents with recent code changes as a way to further improve RCA precision.

For RecoverIT, this paper is particularly relevant because it demonstrates the practical combination of multi-agent orchestration, cloud monitoring, Kubernetes analysis, source-code inspection, automated remediation, pull-request generation, human approval, and auditability.

2.3 LLM-BASED RCA AGENTS

Roy et al. (2024) investigate the use of Large Language Model agents for automated Root Cause Analysis of cloud incidents. The paper focuses on the ReAct agent

framework, in which an LLM alternates between reasoning and tool-use actions to investigate an incident. The central idea is that an RCA agent should not be limited to the information already contained in an incident report; it should be able to retrieve additional information and interact with diagnostic tools in a manner similar to an on-call engineer. The authors build their RCA agent around the ReAct framework. Each iteration consists of one reasoning step which yields the next action, tool selection from a predefined action space, observation of the tool's output, and reasoning once again.

The process ends either at reaching a root cause of the problem or when the maximum number of iterations is reached. The model relies on LangChain and the authors constrain the ReAct loop to 20 iterations due to limited resources. The first set of experiments evaluates the performance of the agent in a generalized setup with no access to the team's diagnostic services. The available tools include incident-details tool, which allows for querying the incident description and the historical-incidents tool which returns previous similar incidents. The goal of this experiment is to see how good is the agent in performing RCA with only general information sources.

There are two kinds of retrievers used in the framework for historical event retrieval. Dense retriever is designed using the SentenceTransformers all-mpnet-base-v2 model and Max Marginal Relevance, while the sparse retriever uses the BM25 technique. The agent can generate retrieval queries during its reasoning process. One retrieval variant directly returns retrieved historical incidents to the agent, while another uses a two-step process in which the agent generates a query and then performs question answering over the retrieved incidents. The two-step approach helps separate the retrieval query from the target incident and reduces context-length problems.

Authors make use of a massive internal incident dataset that is gathered through an enterprise incident portal during January 2020 and September 2021. This dataset consists of around 107,000 unique incidents split into training, evaluation, and test sets. In the experiments, 100 evaluation incidents and 500 test incidents are chosen randomly. The historical incidents from the training dataset constitute the retrieval corpus while incident title and description serve as the main source of information for target incidents. GPT-4 with 8,000-token context window is selected as the main language model while GPT-3.5-Turbo is selected to summarize incident descriptions, root causes, and discussion comments in order to decrease context window size and costs.

The ReAct agent is compared with several non-agent baselines such as Retrieval Baseline, zero-shot Chain-of-Thought, and Interleaving Retrieval with Chain-of-Thought. These baselines allow for comparing retrieval, explicit reasoning, and agent-based reasoning approaches. Evaluation includes automatic and qualitative measures. Automatic evaluation makes use of BLEU, METEOR, ROUGE-L, and BERTScore metrics in order to compare generated root causes with reference answers. Since these metrics do not assess factual correctness completely, manual annotations of predictions are provided using categories such as precise, imprecise, hallucinated, lack of evidence, reasoning error, and retrieval error.

This qualitative evaluation is important because an RCA system can generate a response that appears semantically similar to the reference while containing unsupported or misleading details. In the generalized evaluation, the retrieval baseline improves on several lexical metrics as more historical incidents are retrieved, while semantic similarity changes only slightly. The ReAct variants retrieve approximately four unique historical incidents per incident on average, with about two retrieval lookups. The qualitative analysis shows that the ReAct approach has lower hallucination rates than the retrieval-only baseline and can sometimes filter out historical incidents that are lexically similar but diagnostically different. However, the ReAct agent also produces cases where it cannot determine a root cause because the available incident information is insufficient.

The authors also investigate whether discussion comments attached to historical incidents improve RCA. These comments contain records of diagnostic steps taken by on-call engineers. The experiments show mixed changes in lexical metrics across models, while semantic metrics indicate that adding discussion comments does not significantly change the resulting root-cause predictions. The authors explain that discussion comments often contain administrative information or diagnostic steps that depend on tools unavailable to the agent, limiting their usefulness when the agent cannot reproduce the same investigation process. The second major part of the paper is a practical case study with the Azure Fundamental Team.

Instead of using only generalized retrieval tools, the researchers adapt the ReAct agent to interact with team-specific diagnostic services. They work with the team over several weeks to understand its RCA process, identify the diagnostic services used by engineers, and develop tools that allow the agent to interact with those services. The

case-study agent is given access to several specialized tools. The Database Query Tool helps in forming queries, executing those queries on the database system of the team and analyzing the results received from it. The results are transformed into Pandas DataFrames and subsequently an LLM-powered DataFrame Query Engine is used to answer questions related to those results.

This allows the agent to perform both database retrieval and subsequent interpretation of structured results. The KBA Question-Answering Tool is provided to the agent in order to extract information from the knowledge base articles. The knowledge base articles consist of troubleshooting techniques, database queries, systems information, and other operational knowledge utilized by on-call engineers.

2.4 LOG-BASED RCA

LogRCA focuses on root cause analysis in distributed services using log data as the main source of diagnostic evidence. Its relevance to RecoverIT comes from addressing one of the most important problems in automated incident investigation: operational logs contain large amounts of information, but only a small portion may be useful for identifying the underlying cause of a failure. The work therefore supports the project's need for an evidence-selection and evidence-analysis stage rather than simply passing all available logs to an AI model. For the proposed system, logs will be one of several evidence sources used by the incident investigation agent. When an alert is received, the agent should be able to retrieve logs related to the affected service and incident time window, identify the relevant error patterns and connect them with other operational information.

LogRCA is therefore particularly relevant to the log-analysis component of the proposed architecture and to the broader objective of reducing the amount of irrelevant information presented to the reasoning layer. The paper's distributed-service focus also aligns with RecoverIT. However, in the case of cloud native or microservice architecture, a single issue can produce logs in multiple services and components. The error which is visible may have happened in a downstream service but the initiating issue could have occurred somewhere else. It is therefore important to look into the relation between the log and the timeline of the incident.

In case of cloud-native or microservice systems, a single failure could result in multiple logs being emitted from different services and components. An obvious symptom of

the failure could happen in one of the downstream services, whereas the initial failure has happened somewhere else. It means that we need to look at the connection between log events and the whole incident timeline rather than looking at the root cause based on a single error message.

For the original project proposal, the primary learning is that the log analysis has to be combined with all other evidence. The agent could use the logs for finding symptoms and potential root causes but then use Git history, CI/CD pipeline, deployment history, metrics and service health status for validation of these root causes. For instance, a database connection error found in the logs would be checked for the fact that the database is really unhealthy, for the recent changes in database configuration, for the correlation of the appearance of such failures and the last deployment. All this is in line with our proposed investigation process. Instead of having the LLM infer a root cause from the raw logs, the system would first gather the appropriate log window, extract events of interest, create a timeline and then use these events together with evidence collected from software delivery and runtime environment..

The resulting incident context can then be passed to the root-cause reasoning component. The paper is therefore relevant mainly to RecoverIT's observability and RCA stages. It establishes a foundation for dealing with logs as structured data that have to be analyzed and filtered in order to be able to use it as input for root cause analysis. Log analysis in the context of the whole project is one of the possible tools available to the investigator, along with Git, CI/CD, deployment, metrics and service health.

After evidence is collected and a root cause is identified, the system can generate a controlled remediation plan, execute an allowed recovery action when appropriate, and verify whether the incident has actually been resolved. Thus, LogRCA contributes to the evidence-analysis foundation of the proposed system, while the project combines that capability with agentic investigation, CI/CD context, safe remediation and closed-loop verification.

2.5 MICROSERVICE RCA SURVEY

This survey provides a broad overview of Root Cause Analysis in microservice environments and is highly relevant to RecoverIT because it establishes the different types of operational evidence that can be used to investigate incidents. It organizes RCA

approaches around metrics, traces, logs, multi-modal data and the emerging role of Large Language Models. This is directly related to RecoverIT's objective of allowing an AI agent to collect and correlate evidence from multiple sources instead of relying on a single alert or log message. The survey describes RCA as a process that begins with data collection, followed by analysis of the collected evidence and then formulation and validation of possible root causes. This structure closely matches RecoverIT methodology.

When an incident is received, our system should first gather relevant operational and CI/CD information, analyze the evidence to identify patterns and relationships, and then generate and evaluate possible root-cause hypotheses. The survey's discussion of metrics is relevant to the monitoring component of the proposed system. Metrics such as CPU utilization, memory consumption, network activity, request rates and latency can indicate which service or component is affected. For the original project proposal, these metrics can be used to determine whether an incident is related to resource exhaustion or service degradation and can also be used after remediation to verify whether the system has recovered. The discussion of traces is relevant to distributed-service investigation.

In a microservices-based system, the requests from users can traverse multiple services; hence, it becomes tricky to pinpoint the root cause of the fault. The trace information would assist in tracing the path of an impacted request and correlating faults across dependent services. The discussion of the use of logs by the survey also plays a crucial role in the original project proposal. Log files hold rich information about various application and infrastructure events, but it is not easy to analyze due to high volumes and differences. The incident agent proposed can thus fetch the pertinent window of log files and correlate the critical error events along with other operational data.

A particularly important part of the survey for RecoverIT is its discussion of multi-modal RCA. The survey points out that metrics, traces, and logs give different perspectives on the same event. This combination gives more powerful proof than just using one of them. Our proposed system extends this multi-source idea beyond runtime observability by also considering Git changes, CI/CD pipeline results, deployment history and configuration changes. This allows the agent to connect an operational failure with events in the software-delivery lifecycle.

The survey also discusses the role of LLMs in RCA. This supports the proposed use of an LLM as the reasoning and orchestration component of the incident agent. Rather than expecting the LLM to infer a root cause from a fixed prompt, the proposed system can use it to decide what evidence should be collected, call controlled diagnostic tools, compare evidence and generate explainable root-cause hypotheses. The survey identifies several challenges that are directly relevant to RecoverIT, including complex dependencies between services, propagation of failures across a system, noisy and heterogeneous operational data, and the difficulty of accurately localizing the original fault. These challenges justify the project's emphasis on timeline construction, evidence correlation and hypothesis-based investigation.

Overall, this survey provides the conceptual foundation for the RCA portion of RecoverIT. It supports a methodology in which the incident agent combines multiple evidence sources, reasons about relationships between events and services, identifies likely root causes and uses the resulting diagnosis as the basis for a controlled recovery action. RecoverIT places this RCA capability inside a larger CI/CD incident-response workflow that continues through remediation and post-action verification.

2.6 CAUSAL MICROSERVICE RCA

In this paper, the Root Cause Analysis in relation to microservices is investigated from the angle of causal inference. The importance of this work for RecoverIT can be seen in its solution of one of the most important problems in incident analysis, namely the fact that the component exhibiting some abnormal behavior is not always the component that triggered the incident. Since in microservice systems incidents can spread across services via service dependencies, the establishment of a relationship between two incidents does not necessarily prove their causality. This point is very important for RecoverIT, since the incident agent should analyze the underlying cause of the incident rather than report on the service that generated the largest number of errors. For instance, the introduction of wrong configuration in one service by deployment could lead to database or dependent services' failures.

A causal perspective encourages the investigation to consider the dependency structure and the direction of the failure rather than treating every correlated signal as an equally likely root cause. The paper is therefore relevant to the proposed system's root-cause-analysis module and its use of evidence from distributed services. The agent can

combine operational observations with information about service relationships, recent changes and incident timing to determine which component or change is more likely to have initiated the failure. This complements RecoverIT's timeline-based investigation, where events such as a Git commit, build, deployment, configuration change and subsequent service degradation are ordered to help distinguish possible causes from downstream effects. The paper also highlights that causal RCA methods have different assumptions and strengths and that no single causal approach can be expected to perform equally well for every microservice failure scenario.

It is crucial to this project proposal for the agent designed to use the proposed RCA algorithms to not depend exclusively on one single RCA algorithm. Causal reasoning can be used in combination with log data, metrics, traces, deployment history, Git diffs, and CI/CD as part of an evidence-based approach. This ties into the hypothesis-based reasoning planned for the project. The agent will be able to produce different hypotheses concerning the problem in question and use further evidence to confirm which one of these hypotheses fits the order of events better. Causal information will make this process easier by allowing the agent to identify the true cause of the event among subsequent symptoms caused by dependencies.

It is also related to the RecoverIT evaluation technique, as it proves that RCA methods must be checked under various types of failures instead of taking for granted that an efficient RCA approach for one type of incident will be effective for all others. Therefore, for our controlled prototype, it justifies the usage of various incident scenarios, namely deployment regression incidents, configuration incidents, service incidents, and dependency incidents. In relation to RecoverIT project, its primary significance lies in the causality aspect of RCA.

With the help of the described combination of this perspective with multi-source evidence gathering and CI/CD context, it becomes possible to use the gained diagnosis for selecting a controlled remediation action and verifying if the incident was handled.

2.7 RCA BENCHMARKING

Pham et al. (2025) present RCAEval, an open-source benchmark aimed at solving one of the key problems associated with RCA for microservices: the absence of large-scale and reproducible data as well as the absence of an evaluation framework. Specifically, the authors note that there is a tendency among researchers to evaluate their techniques

in terms of performance on a few systems and few kinds of faults, which makes a direct comparison of different approaches very difficult and does not allow researchers to gauge the generalization ability of their approaches. RCAEval focuses on shifting the attention from developing another RCA algorithm to creating a common ground upon which different RCA algorithms can be created, compared, and reproduced. RCAEval includes 735 cases of failures in three microservices: Google Online Boutique, Weaveworks Sock Shop, and Train Ticket. What is important, the benchmark includes multiple telemetry modalities: metrics, logs, and traces.

These datasets include RE1, RE2, and RE3. RE1 consists of metric-based failures, whereas RE2 is an extension of RE1 by taking into account multi-source RCA based on metrics, logs, and traces. RE3 considers code-based failures, while providing multi-source telemetry. Every failure instance includes information about the root-cause service as well as the indicator for the root cause such as a certain metric or log that helps in conducting both coarse and fine grain localization. Thus, the benchmark can provide a much more comprehensive basis for RCA evaluation than those which rely on a single telemetry source or synthetic data.

The paper is particularly useful for RecoverIT as the project approach is also focused on the multi-source evidence correlation approach for incidents analysis. The following evidence sources are listed in the project specification: application and infrastructure logs, logs from CI/CD pipelines, monitoring metrics, deployment history, Git commits, pull requests, configuration changes and information about the health of services. RCAEval demonstrates the significance of correlating heterogeneous operational evidence rather than relying on one signal. The use of metrics, logs, and traces makes it possible to build an excellent base for the telemetry analysis layer of the project. In addition to that, the project focuses on Git history, deployment and CI/CD history and broadens the context of incidents to the software delivery life cycle.

In distributed microservice systems, a fault in one component can propagate through service dependencies and create symptoms in several downstream services. Consequently, the most visible anomaly is not necessarily the initiating fault. This is closely aligned with the proposed agent's investigation workflow, where the system is expected to construct a timeline, generate multiple root-cause hypotheses, and evaluate supporting and contradicting evidence instead of immediately selecting the first observed error. RCAEval's annotated root-cause indicators provide an appropriate

reference for evaluating whether an autonomous investigator can identify the underlying cause rather than merely detect abnormal services. Another important contribution of RCAEval is its reproducible evaluation framework.

The paper provides fifteen baselines spanning metric-based, trace-based, and multi-source RCA and supports evaluation at both coarse-grained and fine-grained levels. This is important for RecoverIT because an autonomous incident agent should be evaluated systematically rather than only through qualitative demonstrations. RCAEval suggests a model for controlled experiments in which known faults are injected, telemetry is collected, and the predicted root cause is compared with ground truth. the design could adopt this evaluation philosophy while extending the measured outcome from RCA alone to the complete incident-response workflow. The benchmark's scale also highlights the practical difficulty of RCA. RE2 contains very large quantities of telemetry, including millions of log lines and millions of traces for the benchmark systems.

This supports an important design principle for the proposed agent: it should not blindly pass all available operational data to an LLM. Instead, it should selectively retrieve and summarize evidence relevant to the incident. This connects directly with the project's Incident Context Builder and Data Collector, where the agent is intended to determine what information is missing, query relevant data sources, and construct a structured incident timeline. The same evidence-reduction principle is visible in other supplied literature: LogRCA seeks a minimal set of causally relevant log lines, while MicroRCA-Agent applies filtering and summarization before multimodal reasoning. RCAEval's inclusion of code-level faults is especially relevant to a CI/CD-oriented incident trigger.

The RE3 benchmark is based on faults inserted into the application's source code, and utilizes telemetry like logs and traces for their diagnosis. This illustrates that symptoms observed in production systems may be traced back to the problems in code rather than problems in the infrastructure alone. For RecoverIT, this implies that there is a need to correlate telemetry with recent Git check-ins, pull requests, deployments, and configuration changes. As such, a CI/CD incident can be understood as a series of events starting from the change in the source code to deployment and finally to symptoms, and not just as an isolated incident detected through monitoring.

Wang and Qi's survey organizes microservice RCA into metric-based, trace-based, log-based, multi-modal, and LLM-assisted approaches, emphasizing the need to analyze heterogeneous operational evidence. Roy et al. show that LLM agents can improve RCA by dynamically retrieving additional diagnostic information rather than relying only on an incident description. Flow-of-Action further demonstrates that unconstrained LLM agents can make irrelevant or hallucinated tool calls and therefore benefit from structured procedures, auxiliary agents, and controlled action selection. These findings are directly applicable to the proposed system's design: evidence collection should be tool-driven, reasoning should be explicit, and autonomous actions should be constrained by safety policies.

2.8 AUTONOMOUS CLOUD EVALUATION

The work by Chen et al. (2025) introduces AIOpsLab as an integrated design, development, and evaluation framework for AI agents that support autonomous cloud operations. The motivation behind the work is attributed to the rise in complexity of cloud and microservices environments, together with the lack of capability of existing AIOps benchmarks in assessing AI agents on the entire lifecycle of incidents. Instead of assessing an AI agent either on static datasets or operational task, AIOpsLab provides an interactive environment where the agent could interact with the running cloud system by observing, investigating faults, taking actions, receiving feedback, and getting evaluated against a predetermined expected result. The wider paradigm introduced in this paper is referred to as AgentOps, which allows AI agents to conduct various cross-layer operations rather than detection/classification task.

The system includes a microservice cloud infrastructure, a workload generator, fault injection capability, telemetry capture mechanism, an orchestrator, and the Agent-Cloud Interface (ACI). ACI acts as a control interaction interface through which the agent can interact with the cloud environment, perform actions, and receive feedback from the cloud. Thus, the incident response scenario in this framework involves a sequence of interactions rather than a single step prediction, which has high relevance to RecoverIT as RecoverIT also involves the same loop for investigation: incident arrives, evidences collected, hypotheses formulated and tested, recovery plan/action made and system state verified. In AIOpsLab, the operational problem is modeled as P = <T, C, S> where T stands for task, C for context and S for oracle.

The task taxonomy comprises four levels: detection, localization, RCA, and mitigation. Detection is the determination of the existence of a fault; localization determines which service or component has been affected; RCA detects the layer of the system at which the fault occurred, and the type of the fault; while mitigation involves interaction of the agent with the environment to fix the fault, such as fixing the configuration or reverting to an earlier version. This hierarchy is closely aligned to the RecoverIT framework process starting from detection and triage to RCA and then remediation. In addition, it reinforces the theory that incident management should be judged as a series of increasingly difficult operational tasks rather than on the basis of RCA scores. The evaluation framework uses two microservices applications from DeathStarBench as case studies.

The tools that have been used for metrics, distributed tracing, and logging are Prometheus, Jaeger, and Filebeat/Logstash respectively. The Kubernetes and Helm API is part of the orchestrator. This multi-source telemetry design is directly relevant to RecoverIT because our incident investigator is expected to reason over logs, metrics, service health, deployment information, and other operational evidence. AIOpsLab demonstrates how an agent can interact with telemetry in a live environment instead of receiving a fixed, pre-collected incident record. Another important contribution is the benchmark itself.

The paper proposes 48 evaluation problems that focus on various AIOps use cases and evaluates multiple LLM-based agents. Unlike previous benchmarks, which focused on incidents involving only performance degradation or application crash, the proposed one uses more sophisticated scenarios and precise root causes. The evaluation framework takes into account not only accuracy but also efficiency metrics like Time-to-Detect (TTD), Time-to-Mitigate (TTM), number of agent actions/steps, and token consumption. This particular evaluation method is relevant to RecoverIT as our intended evaluation not only should show that the agent has identified the root cause accurately but also should take into account the time and efficiency of investigation, success of the remediation process, and number of actions performed.

The paper reports that the RCA and mitigation tasks are the most challenging for the evaluated agents, and that none of the agents consistently achieves high performance across all four task categories. The authors also show that increasing the number of agent steps can improve performance for some agents, but that performance may

eventually plateau. In the reported experiments, FLASH reaches its highest accuracy of 59.32% with a step limit of 20, while increasing the step limit for GPT-3.5-Turbo beyond a certain point mainly increases token consumption without improving accuracy. These findings support RecoverIT's emphasis on purposeful investigation rather than unrestricted agent exploration. AIOpsLab also identifies failure modes that are important for the proposed system.

Agents can struggle with complex operational tasks, use unnecessary actions, misinterpret normal system behaviour as an incident, and have difficulty completing mitigation. The paper reports false-positive detection behaviour on no-fault scenarios and notes that some agents may produce explanations that are inconsistent with the actual fault or workload. The use of an Agent-Cloud Interface as a guard-rail and the use of predefined evaluators are therefore important design ideas. All these notions are consistent with the safety layer implemented by RecoverIT, where the AI system should not be given unfettered access to infrastructure and remediation should go through policy, allowlist, approvals, dry run, and post-execution check. Moreover, this paper has significance from an evaluation standpoint since it asserts that static telemetry data sets and static Q&A benchmarks do not capture dynamic cloud activity well.

A real incident changes the state of the environment as investigation actions are performed, and a mitigation action can itself change the subsequent evidence available to the agent. AIOpsLab addresses this by allowing agents to interact with an environment and by evaluating the final system state.

2.9 SOP-GUIDED RCA

Pei et al. (2025) propose Flow-of-Action, a Standard Operating Procedure (SOP) enhanced LLM-based multi-agent system for root cause analysis in microservice environments. It should be noted that the paper highlights a significant drawback of general-purpose LLM agents that can be applied to RCA, despite the fact that the ReAct paradigm is similar to how Site Reliability Engineers analyze issues; an LLM may hallucinate, choose inappropriate tools, and even be overloaded by various and complicated observations. These errors can influence subsequent investigation steps and lead the agent away from the actual root cause. Flow-of-Action therefore focuses on constraining and structuring the investigation process rather than allowing the LLM

to select every action freely. The central idea is to encode experienced SRE diagnostic procedures as Standard Operating Procedures and use them as guidance during RCA.

The authors introduce an SOP-centric framework called SOP flow. It contains tools for finding the SOPs relevant to an incident, automatically generating an SOP when a suitable procedure is unavailable, and converting SOPs into executable code.The SOP serves as a soft constraint and directs the main agent to the correct trajectory for diagnosis, while also having room for flexibility in case something unforeseen occurs. This is important for RecoverIT because the project also requires an agent to investigate incidents dynamically, but within a controlled and explainable workflow rather than through unrestricted LLM reasoning. Flow-of-Action also changes the action-selection process.

Instead of immediately choosing one tool or action after every observation, the system introduces a thought-actionset-action-observation paradigm. The agent first reasons about a set of plausible actions and then selects an appropriate action. It helps decrease ineffective calls of various tools in case there is more than one way to act after making an observation. For RecoverIT, this principle can be applied to the investigation stage: after receiving an incident, the agent can identify the information it needs such as logs, deployment history, Git changes, metrics, or service health and select the most relevant next investigation step rather than blindly calling every available tool. The paper further divides responsibilities among multiple agents.

The MainAgent coordinates the RCA process, while CodeAgent assists with code-related investigation, JudgeAgent helps determine whether the root cause has been identified, ObAgent extracts important information and fault types from large amounts of observational data, and ActionAgent assists with generating and selecting action sets. This separation of responsibilities is directly relevant to RecoverIT's agentic architecture. Our system similarly benefits from specialized tools and modules for log retrieval, Git analysis, CI/CD analysis, deployment history, root-cause reasoning, recovery planning, and verification. Flow-of-Action demonstrates that distributing complex RCA responsibilities can reduce the cognitive burden on a single LLM and improve the overall investigation process. A major contribution of the paper is its treatment of information overload.

RCA environments can expose agents to large amounts of logs, metrics, traces, code information, and other observations. ObAgent is designed to extract fault types and key information from this data before it is used by the main agent. This is closely connected to RecoverIT's Incident Context Builder and Data Collector. Although the project specification explicitly suggests collecting logs, metrics, CI/CD information, Git changes, deployments and configuration changes, the agent must not simply report all unprocessed facts to the LLM. Rather, the evidence must be filtered, organized and correlated into a timeline suitable for root cause analysis.

The paper's approach is also relevant to explainability and repeatability. SOPs represent explicit diagnostic knowledge: they describe why particular investigation steps should be performed and provide a structured path through an incident. In RecoverIT, a similar idea can be applied when defining remediation and investigation policies. For instance, an incident related to the deployment could lead to the execution of an investigation routine that would check the deployment time line, recent commits, configuration changes, application issues, and service status before suggesting a rollback action.

The experimental results demonstrate the value of these constraints. The paper reports that the ReAct baseline achieves 35.50% RCA accuracy, while Flow-of-Action achieves 64.01%. The authors attribute the improvement to the combination of SOP-guided orchestration, the action-set mechanism, and multi-agent collaboration. Ablation results further indicate that removing individual agents reduces accuracy to some degree, while removing the action-set mechanism has a particularly noticeable effect on action efficiency. The paper also limits the number of investigation steps to prevent unbounded loops.

These findings are highly relevant to the proposed system because they show that autonomous RCA should be designed around controlled reasoning, bounded tool usage, and explicit stopping criteria rather than unlimited agent exploration. Flow-of-Action also complements the other supplied literature. RCAEval provides a benchmark for evaluating RCA using metrics, logs, and traces across multiple microservice systems, while AIOpsLab evaluates agents in an interactive cloud environment where they can detect, localize, diagnose, and mitigate faults. LogRCA emphasizes selecting useful log evidence from very large log collections, and the LLM-agent RCA work demonstrates the importance of allowing an agent to dynamically retrieve diagnostic information.

Flow-of-Action contributes the additional idea that this tool use should be guided by structured operational knowledge and coordinated among specialized agents.

Together, these works support RecoverIT's architecture of evidence collection, structured reasoning, controlled action selection, and verification. For the Autonomous CI/CD Incident Triager & Self-Healer, Flow-of-Action is especially relevant to the investigation and decision-making layers.

2.10 INTELLIGENT FAULT DETECTION

Ji and Luo (2025) propose a cloud-based AI framework that combines Large Language Models (LLMs) with machine-learning-based fault detection to support intelligent fault detection and autonomous self-healing in cloud systems. The motivation behind this paper comes from the growing size and complexity of cloud computing infrastructure where a rule-based detection mechanism as well as human intervention becomes increasingly hard to achieve. The suggested framework uses the semantic capability of an LLM in processing system logs and error reporting as well as real-time data streams, whereas traditional machine learning approaches have been used for fault classification and anomaly detection. The framework follows a multi-level architecture.

The method uses supervised learning for fault classification along with unsupervised learning for detecting anomalies, thus enabling the framework to detect abnormal behavior and classify fault conditions. The LLM module helps understand semantics of the operational data like logs and error messages, while the machine learning modules enable structured detection capability. Once the fault is detected, a self-healing process is automatically initiated by the framework. This technique is applicable to RecoverIT since our framework also requires a combination of analysis of the operational data along with higher reasoning apart from just using the rule-based approach. One of the important parts of the paper includes unstructured operational data.

There are cloud events that produce text-based artifacts such as logs, error messages, alerts, and reports that are challenging to analyze effectively using purely numeric anomaly detection methods. The paper uses LLM natural-language understanding to interpret this information semantically. For RecoverIT, this supports the use of an LLM as the reasoning layer over heterogeneous operational evidence. Application logs and CI/CD logs can contain different descriptions of the same underlying incident, while Git commits, deployment changes, configuration changes, and alert messages may

provide additional context. An LLM can help transform these heterogeneous observations into a structured incident context that can then be used for root-cause reasoning.

The research paper further makes a comparison of various approaches used for fault detection. These include Transformer fault detection, Graph Neural Network (GNN) based fault detection, and Deep Reinforcement Learning (DRL) for autonomous self-healing. This is significant for RecoverIT because it demonstrates that fault management is not a single-model problem. Various methods may be effective for different phases. For instance, anomaly detection methods may detect unusual activities, graph approaches may help in understanding the connections in distributed systems, reinforcement learning may assist in making decisions during recovery process while LLMs may help in understanding unstructured operational information. RecoverIT similarly separates incident detection, evidence gathering, root cause analysis, recovery planning and verification rather than trying to create one model to do everything.

Self-healing systems are meant to automatically react to faults through corrective measures such as service restart, resource allocation, or corrective changes. According to Ji and Luo, detection alone is insufficient since for a practical reliability system in the cloud environment, downtime needs to be reduced by recovery in addition to just detection. The approach adopted by Ji and Luo connects fault detection to the process of automatic self-healing. In relation to RecoverIT, this refers to the use of the Recovery Planner and the Safe Action components, which involve translating the suspected cause into a deliberate recovery operation such as rollback, restart, scaling, and retry.

Rather than going from an anomaly straight into a recovery action, our agent is designed to establish a timeline of the incident, gather more evidence, formulate multiple theories regarding the root cause, rate the theories based on supporting and conflicting evidence, and only then choose a recovery action. The Ji and Luo framework therefore provides a useful foundation for the detection-to-self-healing relationship, while RecoverIT extends the workflow toward explicit agentic investigation and evidence-based RCA. The paper's use of multiple operational data sources is also closely related to the proposed system's observability layer. the original project proposal is designed to collect application and infrastructure logs, CI/CD pipeline logs, monitoring metrics, deployment history, Git changes, configuration changes, and service-health

information. Ji and Luo specifically emphasize logs, error reports, and real-time data streams as inputs to the LLM-based framework. This supports the project's decision to treat operational information as a heterogeneous evidence space rather than relying on a single monitoring signal.

The paper is complementary to the other supplied RCA literature. RCAEval shows the significance of RCA evaluation from multiple telemetry streams and practical microservices fault cases. On the other hand, LogRCA is dedicated to finding the limited root-cause information among the huge quantity of log data. Roy et al. demonstrate that LLM agents can dynamically retrieve additional diagnostic information during RCA. AIOpsLab evaluates agents in interactive cloud environments and shows that complete RCA and mitigation are more difficult than simple detection or localization.

Flow-of-Action also shows how investigation using LLM requires the use of procedures and limited action selection. These two papers indicate that there is need for the suggested system to consider both detection, evidence gathering, reasoning, action selection and verification rather than taking self-healing to be an automatic reaction to a warning signal. Ji and Luo also note that there is improvement in fault detection compared to conventional techniques. These outcomes are directly related to the evaluation goals of RecoverIT. Our system should not be evaluated only on whether it can generate a plausible root-cause explanation.

2.11 AGENT-BASED MICROSERVICE RCA

Overall, MicroRCA-Agent demonstrates that effective LLM-based RCA in microservice environments depends not only on the reasoning capability of the LLM, but also on the quality, compression, alignment, and integration of the evidence provided to it. Its modular multimodal architecture provides a strong reference for RecoverIT's evidence-driven RCA layer. RecoverIT builds on this direction by adding CI/CD and source-change context, agentic evidence retrieval, hypothesis-based reasoning, constrained remediation, and closed-loop verification, thereby connecting multimodal RCA with the broader incident-response and self-healing workflow. The paper also supports the project's evaluation strategy. MicroRCA-Agent demonstrates the value of ablation experiments for understanding which evidence sources contribute to RCA.

A similar approach can be used in RecoverIT by comparing an agent with logs only, logs plus metrics, logs plus metrics plus traces, and finally the complete CI/CD-aware evidence set including Git and deployment information. In addition to root cause accuracy, the next version of RecoverIT may analyze investigation time, evidence quality, number of investigation steps, remediation accuracy, false remediation and recovery time. In terms of RecoverIT, MicroRCA-Agent will be especially useful for the following modules: Evidence Collection, Incident Context Builder, RCA Engine. The architecture of the project could borrow the modality-specific preprocessing idea from it, i.e. logs could be preprocessed by parsing and filtering; metrics - by anomaly reduction; traces or service health information could be summarized before submitting to the reasoning agent. Then the project broadens the operational context with CI/CD specific evidence, such as recent git commits, pull requests, deployment history, pipeline execution results and configuration changes. Thus, the agent is able not only to understand where the failure happens, but also what has been changed right before that.

The paper complements several other supplied studies. RCAEval provides a repeatable benchmark for RCA of microservices through metrics, logs, and traces, whereas AIOpsLab performs experiments on evaluating AI agents in interactive cloud environments in terms of detection, localization, RCA, and mitigation. LogRCA concentrates on finding causal information in log data, and Roy et al. demonstrate the advantages of LLM agents in dynamically gathering more diagnostic information. Flow-of-Action adds SOP-guided and multi-agent control to reduce irrelevant or hallucinated agent actions. MicroRCA-Agent fits into this research progression by showing how carefully preprocessed multimodal telemetry can be combined with LLM reasoning for practical microservice RCA.

The paper's results also show the importance of managing LLM context and hallucination risk. MicroRCA-Agent uses filtering, summarization, structured prompts, and modular processing to control the information supplied to the LLM. The reported analysis includes cases where LLM reasoning can still produce hallucinations, demonstrating that multimodal LLM-based RCA is not automatically reliable simply because multiple data sources are provided. This is relevant to the proposed system's safety architecture. The project should constrain the LLM through predefined tools,

structured evidence, action allowlists, policy checks, and post-action verification rather than allowing unrestricted infrastructure actions.

MicroRCA-Agent also provides an important lesson about the role of LLMs in operational systems. The use of the LLM is not meant to serve as an alternative to all traditional approaches to monitoring and anomaly detection. Instead, traditional algorithms are applied to the reduction and structuring of the data generated by the operations, and the LLM comes into play when the need for semantic processing and multimodal reasoning arises. This hybrid design is directly relevant to RecoverIT. Indeed, our system may leverage traditional observability tools and deterministic queries to retrieve data and detect anomalies, but the agent will conduct hypothesis generation, evidence correlation, root-cause ranking, and recovery planning.

This division can reduce unnecessary LLM calls and make the system more explainable and controllable. A particularly important finding is the complementary value of the different modalities. The paper's ablation analysis shows that removing or isolating modalities changes performance and that combining modalities can improve RCA quality. In particular, the reported results highlight strong synergy between log and metric information, while the metric component also makes a substantial individual contribution. This supports a central design decision in RecoverIT: incident diagnosis should not rely only on application logs or only on monitoring metrics.

Logs can provide semantic descriptions of failures, metrics can identify quantitative deviations, and traces can show distributed request behaviour. Combining these signals provides a more complete incident context. The Multimodal Root Cause Analysis module is the core decision-making stage. It integrates the processed log features, anomalous trace information, and summarized metric phenomena and uses carefully designed cross-modal prompts to reason over the combined evidence. The output is structured around the faulty component, root-cause description, and reasoning trace.

This directly corresponds to RecoverIT's requirement for evidence-based RCA rather than a simple natural-language answer. Our proposed agent should similarly produce the most likely root cause together with supporting evidence, confidence, and an explanation of why alternative hypotheses are less likely. The Metric Fault Summarization module is another important component. Rather than sending all metric dimensions to the LLM, the system first uses a statistical symmetric-ratio filtering

strategy to identify potentially abnormal metrics and remove normal or redundant information. It then uses a two-stage LLM analysis to summarize phenomena hierarchically across service, pod, and node levels.

This is especially relevant to RecoverIT's Incident Context Builder. Our solution needs to do exactly the same thing, turn monitoring data into the explanation of what has changed, where the changes happened, and how the incident was affected by this. Also, the hierarchical analysis suits well the architecture of cloud native applications, which generate incidents and symptoms on multiple levels at once.

2.12 MULTI-AGENT RCA SEARCH

Naakka, Wang, and Mäntylä (2026) propose LATS-RCA, an LLM-based multi-agent framework for root cause analysis in microservice systems. This paper deals with the challenges of RCA in the manual setting of distributed microservices, wherein engineers have to correlate signals of operations and analyze multiple responsible services within a tight deadline. A weakness in previous works on RCA using LLM is found by the authors: all such approaches employ a linear reasoning strategy, which is inappropriate when the first appearance of an anomaly suggests that there are multiple services/components behind it. This challenge is solved in LATS-RCA through formulating the task of RCA as a tree search guided by reflection. The core of LATS-RCA is the Language Agent Tree Search mechanism, combining LLM reflection with Monte Carlo Tree Search.

Multiple LLM-driven agents iteratively reason over execution logs and performance metrics, while the search maintains competing hypotheses and expands promising diagnostic paths. Reflection evaluates intermediate reasoning states using evidence quality, diagnostic completeness, and internal consistency. This is highly relevant to RecoverIT because the original project proposal also intends to generate several root-cause hypotheses and rank them using supporting and contradicting evidence rather than selecting the first plausible explanation. The architecture contains specialized log and metric agents coordinated by a supervisor. Each agent investigates its own observability modality, while the supervisor controls sequencing, evaluates progress, performs cross-modal correlation, and can hand off a condensed evidence summary when additional analysis is required.

It is highly related to the architecture of RecoverIT in which the various capabilities deal with logs, metrics, Git, CI/CD, deployment history, RCA, recovery planning, and verification. In this paper, the importance of specialization with respect to central coordination is shown. One important aspect in particular is the presence of multiple candidate hypotheses. Instead of focusing on one chain of thought for RCA, LATS-RCA takes into account different alternatives and decides which should be investigated by reflection. It fits in with the RCA Engine proposal in which the increase of HTTP 500 errors might be due to a faulty deployment, database issue, configuration problem, and so on.

The agent can collect additional evidence and update hypothesis confidence before recommending remediation. On the LO2 benchmark, LATS-RCA achieves 91.3% diagnostic accuracy, compared with 39.8% for single-agent ReAct and 57.4% for multi-agent ReAct. The ablation study shows that candidate batching, backpropagation, and reflection all contribute to performance, with removal of candidate batching reducing accuracy to 84.3%. These results support structured hypothesis exploration over simple linear reasoning. The accuracy improvement comes with substantially higher computational cost.

LATS-RCA averages 53.1 API calls, 156K tokens, and 9.1 minutes per investigation, compared with 13.5 calls, 41K tokens, and 1.2 minutes for single-agent ReAct. It explores an average of 18.9 hypotheses per investigation. This is important for the original project proposal because an incident-response agent cannot simply maximize reasoning depth; investigation needs bounded tool-call, token, time, and stopping budgets. The production validation is especially relevant. LATS-RCA was deployed in a polyglot production microservice environment with Node.js, Rust, Python, and Go services and evaluated on 37 real-world incidents.

The accuracy level was around 60–70%, which is less than 91.3% achieved for LO2. The production environment involved diverse logs, timestamps, severities, metric names, and coverage. Therefore, operational heterogeneity has a huge impact on agentic RCA. That is why one of the main principles of RecoverIT's design is normalization and correlation of evidence before reasoning. Logs, metrics, Git commits, deployments, and CI/CD outputs have to be converted into the unified incident scenario and timeline. This solution can be extended even further by correlating source code changes and deployments with production symptoms.

The paper complements RCAEval, AIOpsLab, Flow-of-Action, MicroRCA-Agent, and other supplied literature. RCAEval offers reproducible multi-source RCA evaluation; AIOpsLab tests agents in detection, localization, RCA, and mitigation; Flow-of-Action constrains LLM agents through SOPs and action selection; and MicroRCA-Agent shows modality-specific data preprocessing followed by multimodal LLM reasoning. These works confirm that an agent-based architecture that relies on evidence gathering, specialized reasoning, actions, and verification is justified. Regarding the Autonomous CI/CD Incident Triager & Self-Healer, LATS-RCA is particularly applicable for the AI Incident Analyzer and RCA Engine. The design may include hypothesis-based reasoning while expanding the evidence space to Git commits, pull requests, CI/CD pipeline outcomes, deployment history, configuration changes, and service status. This would allow the agent to make conclusions not only regarding the location of the incident, but also regarding the recent change that caused it.

The paper also informs the project's evaluation strategy. In addition to root-cause accuracy, the project should measure investigation time, tool/API calls, tokens, number of hypotheses explored, remediation success, false remediation, and recovery time. This is important because LATS-RCA demonstrates that higher diagnostic accuracy can require substantially greater computational cost. In summary, Multi-Agent Systems for Root Cause Analysis in Microservices shows how using structured multi-agent searches for competing hypotheses can greatly benefit RCA relative to linear analysis, along with its associated costs and heterogeneity problems. It is an excellent basis for the hypotheses generation and reasoning layer of RecoverIT, although the initial project proposal ties the reasoning to CI/CD-based investigation, constrained remediation, and verification in the loop.

2.13 GRAPH-AUGMENTED RCA

Tian et al. (2026) present GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in Microservices. The paper focuses on solving three interconnected issues in microservices RCA: first, the actual root cause might differ from the service in which the problem symptom is found; second, different telemetry modalities might find different potential root causes; third, merely locating the failing service would not suffice for the SRE, who will need a causal explanation and a concrete way to respond. As such, GALA integrates service dependency graphs, multiple types of telemetry modalities, statistical hypothesis formation, and agentic

investigation using the LLM. This makes GALA extremely relevant to RecoverIT in terms of correlating diverse operational data and progressing from detection to diagnosis and eventually controlled recovery. In GALA, metrics, logs, and distributed traces serve as complementing pieces of evidence.

Metrics are numerical measurements (like CPU usage, latency); logs are timestamped events; traces record inter-service calls and their time relationships. The paper builds a service dependency graph using distributed traces in order to allow the agent to reason about the failure propagation topology. This is directly relevant to RecoverIT because an incident may appear in one component while the actual fault originates elsewhere. In our system, the same principle can be extended beyond runtime service dependencies to include relationships between CI/CD pipelines, deployments, Git changes, configuration changes, and affected services. GALA+ uses a four-phase architecture.

Phase I conducts early root cause hypothesis generation through complementary metric and trace-based signal sources. Phase II constructs pod-level diagnostic packages that include pertinent telemetry while maintaining temporal and dependency context. Phase III involves graph-driven agentic investigation, and Phase IV summarizes the entire investigation process into a diagnosis, incident report, and action items.

This staged approach is consistent with the workflow of RecoverIT from incident detection, data collection, AI analysis of incidents, RCA, recovery planning, safe actions, and post-incident reporting. A major contribution is the use of dual-signal hypothesis generation.

GALA+ uses BARO for coarse metric-based ranking and introduces STRIX for trace- and graph-structure-aware ranking. The two rankings are fused into a consolidated candidate list before LLM agents perform deeper investigation. The reason why the log has not been considered for the first ranking phase is that there could be instances when failure would be quite silent and therefore would lead to low-quality error logs; logs can be used in the diagnostic process only later on. This is an important approach to the project's original design in which no evidence can be over-represented or dominating in the investigation process. The pod-centric diagnostic bundle is another important component.

Raw telemetry is distilled into a compact representation that preserves temporal patterns, dependency context, and error signals. This reduces the amount of information

that must be processed by the LLM and allows the agent to focus on evidence relevant to each candidate service or pod. This directly supports our proposed Incident Context Builder, which needs to collect and compress logs, metrics, CI/CD information, deployment history, Git changes, and configuration changes before passing them to the reasoning layer. The graph-guided investigation mechanism is particularly relevant to RecoverIT's agentic reasoning. Branches of investigations are created by GALA+ for the top-ranking candidates simultaneously.

An Investigator Agent assesses a candidate based on the diagnostic package and comes up with an assessment consisting of confidence, evidence, cause, and supporting and contradicting factors. In case of insufficient confidence, the agent investigates further via the dependent services in batches. This way, the search process becomes constrained: the agent will not check all services at random; it will go along the most probable fault-propagation routes and stop once it gathers enough evidence. The method corresponds to the RecoverIT need for generation of several hypotheses regarding the root cause and their ranking based on evidence. For example, in the case when some problems happen after a deployment process, the problem can be a result of the following factors: poor code change, configuration change, database problems, resources depletion, or dependency problems.

Instead of choosing the best option immediately when there is an anomaly in the system, our agent is able to maintain multiple hypotheses, collect facts for all of them, and choose the best one depending on the dependencies and the timing. GALA+ also demonstrates why graph-guided exploration can be preferable to unrestricted LLM tool use. It is indicated in the paper that the previously proposed GALA approach had the possibility to acquire inadequate data due to the fact that it was studying only one telemetry packet at a time, whereas unbounded ReAct search could go into an irrelevant direction, deviating from the possible root causes. The graph therefore functions as a reasoning constraint rather than merely as a retrieval structure.

The results show strong performance. In the Online Boutique and Train Ticket datasets, GALA+ performs at an AC@1 of 74.44% and 73.33% and beats the strongest LLM-based baselines by more than 25 percentage points. Another dataset that is examined in the paper is AegisLab, which features 25 failure modes, and in which GALA+ reports the highest MRR, AC@1 and AC@3 scores among all tested approaches. This demonstrates the merit of leveraging graph-based structure, multimodal cues, and

agential investigation as opposed to a single telemetry stream or unrestricted LLM reasoning. The paper does not evaluate only root-cause ranking.

It also evaluates incident summarization and action recommendations using SURE-Score, a human-guided evaluation framework developed with SRE expertise. SURE-Score assesses dimensions related to the usefulness and quality of incident reports, and human SRE evaluation confirms GALA+'s advantage over Flow-of-Action across the evaluated dimensions. This is particularly relevant to the original project proposal because the intended output is not just a root-cause label.

2.14 EXCEPTION-BASED RCA

Mhatre and Shaikh (2025) propose an automated Root Cause Analysis (RCA) system for microservices architectures using Large Language Models combined with a machine-learning-based exception classifier. The paper concentrates on the huge number of business and runtime exceptions in microservice-based applications and the inefficiencies of exception analysis manually. The main contribution of the paper is in developing a hierarchical approach to aggregate, categorize the exceptions and use tuned LLMs to conduct code-aware RCA using runtime stack trace and source code context.This technique is especially applicable for RecoverIT since it helps illustrate the connection between runtime exceptions and application code using LLMs instead of considering any event just another deviation in monitoring data. The process starts from the creation of an exception aggregation layer based on Apache Kafka.

Exception messages raised in various microservices are placed in a Kafka topic and are then passed to an Exception Classifier. This classifier preprocesses error messages and categorizes exceptions into business exceptions and runtime exceptions. Runtime exceptions are then sent to the RCA service. This is helpful because not all exceptions need to go through the same diagnostics process, and by filtering the stream of events prior to passing the information to the LLM, we can avoid unnecessary processing. In the case of RecoverIT, the same technique of staged processing can help separate informational events, application errors, deployment problems, and incidents needing investigation.

The Exception Classifier compares traditional machine-learning approaches including XGBoost, Support Vector Machine, and Naïve Bayes with LLM-based classification. The paper reports that XGBoost performs best among the selected conventional

classification models. The experience from this research may be useful for further work on the proposed system since an LLM doesn't necessarily have to substitute all conventional methods. Simple deterministic algorithms and machine learning models will do the bulk of the classification, and the LLM will be used only for those functions that require reasoning and context understanding. This kind of design follows from the design of the whole system, where operational tools gather data and the AI engine correlates and reasons about it.

The RCA solution is built using CodeLlama and Mistral models that have been fine-tuned. It receives runtime exception stack traces together with tokenized or preprocessed code-repository information and generates a root-cause analysis. The paper reports that CodeLlama-7B performs particularly well when supplied with preprocessed code snippets. The approach therefore establishes an important connection between runtime symptoms and source-code context. For the proposed CI/CD incident triager, this is directly relevant because recent Git commits and pull requests can be used as evidence when an incident occurs after a deployment.

Instead of asking only “which service failed?”, the agent can investigate “what changed in the code or configuration immediately before the failure?” The paper's layered architecture also has a natural correspondence with RecoverIT's workflow. Kafka-based exception aggregation corresponds to the project's Incident Detection and Data Collection layer; exception classification corresponds to initial incident triage; the LLM-based RCA service corresponds to the RCA Engine; and the repository-aware analysis provides a foundation for correlating runtime symptoms with software changes. RecoverIT extends this workflow beyond exception-driven diagnosis by incorporating monitoring metrics, deployment history, CI/CD pipeline results, configuration changes, and service-health information. The biggest merit of the work is in its code-aware approach to diagnostics. While stack traces will show where the exception took place, in order to find out what really happened, it is necessary to analyze the corresponding piece of code and interactions between the functions/components.

Thus, through the use of preprocessed pieces of code, the system tries to shift from exception classification to explaining the cause. It is especially important for RecoverIT since production issues can be caused by a wrong code change despite being manifested as generic HTTP errors, timeouts, and/or services not working even though the

symptoms are clearly observable. The paper states that more than 90% precision in classification of business exceptions, 89.6% recall in runtime exception classification, less than one second per exception for RCA diagnostics, and more than 85% accuracy in human qualitative analysis have been achieved. In this way, the potential of using automatic classification and code-aware LLM diagnostics can be seen. For RecoverIT, speed is especially important.

However, our assessment has to be extended to include the accuracy of multi-source RCA, the investigation process, the success of the remediation, recovery time, and verification reliability. There are a number of research directions that are related directly to the suggested solution. In particular, the authors understand that their existing RCA tool is based on exception stack traces and code context and consider the possibility of using environment logs including resource consumption and network/disk activity. Moreover, they suggest feedback mechanisms for adjusting the model according to changes in the services and mention autonomous remediation as the future direction for the project. All these findings coincide completely with the direction of the initial project proposal, as the suggested agent is supposed to use application and infrastructure logs, metrics, Git commits, deployment logs, CI/CD data, and configuration changes to diagnose problems and move to the next stages.

The paper is complementary to the other supplied literature. RCAEval demonstrates the value of multi-source telemetry and reproducible RCA benchmarks. MicroRCA-Agent combines logs, traces, and metrics through modality-specific preprocessing before multimodal LLM reasoning. GALA uses service dependency graphs and graph-guided investigation to reason about fault propagation. Flow-of-Action introduces SOP-guided multi-agent coordination to control LLM tool use.

Interactive cloud settings are used for evaluating agents for detection, localization, RCA, and mitigation through AIOpsLab. These papers demonstrate a development from specific fault analysis to self-reliant evidence-based investigations of incidents. The contribution of the paper written by Mhatre and Shaikh is significant in providing a software point of view.

2.15 HEALTH-PROBE REMEDIATION

Kalakoti (2025) introduces an AI-enabled self-healing infrastructure architecture that is comprised of continuous health monitoring, incident diagnosis through AI, and predetermined remediation playbooks. This research is concerned with a particular challenge in today's cloud and distributed infrastructure: health monitoring can establish that there is an issue with a particular service, but this is not enough to ensure that the problem will be fixed quickly. Therefore, the solution suggested is a loop that includes health monitoring, incident diagnosis using AI, execution of the predetermined playbooks, and re-evaluation of the situation. This is very much applicable in the context of RecoverIT since it too intends to go beyond simply generating alerts and work towards autonomous recovery. Architecture in this case includes the use of Kubernetes as the underlying infrastructure as well as the inclusion of health probes, monitoring data, reasoning using artificial intelligence, and remediation actions.

The liveness, readiness, and startup probes of Kubernetes offer continuous feedback regarding application health and availability. Where an unhealthy situation is detected, AI-driven reasoning as well as an effective remediation playbook could be applied. It is critical to consider health probes as the triggering approach in RecoverIT since the system requires a means of validating whether the incident is ongoing and whether the remediation operation has indeed brought back the healthy state of service. One major contribution is the integration of AI reasoning together with the predetermined remediation playbooks.

This provides a controlled action space and improves operational safety. For RecoverIT, this principle is particularly important because self-healing actions can have significant consequences. The Recovery Planner should therefore generate or select a recovery plan, but execution should occur through an allowlisted set of safe actions such as restarting a service, rolling back a deployment, scaling a workload, restarting a failed component, or re-running a controlled pipeline step. The structure of the paper can be described as a loop, in which health is monitored, an abnormality is identified, the probable cause is determined with the help of AI-supported analysis, a plan of actions is selected and implemented, and then health is checked to see if the problem has been sorted out. This is very similar to how RecoverIT functions – incident identification, evidence gathering, root cause analysis, recovery planning, action execution and verification of the results.

The key area where these two methods differ lies in the fact that the investigation and correlation aspects become more prominent for RecoverIT. The health probe aspect also shows the significance of objective validation, since a mere assertion of successful remediation made by an LLM is not enough proof of the recovery process. The signals that the system requires include things like service availability, error rate, latency, pod health, deployment information, and application log data. In regard to RecoverIT, this implies that the Post-Remediation Verification module should objectively validate the system state following each remediation action.

If health does not improve, the agent should treat the remediation as unsuccessful and return to investigation rather than terminating the incident automatically. The framework is also relevant to observability. Simple but significant indicators are health checks, while monitoring gives more operational signals. In RecoverIT, the notion can be extended to incorporate logs from applications and infrastructure, metrics, logs from CI/CD pipeline, deployment history, Git commits, pull requests, configuration changes, and information about the service health state. All these can be aggregated in the structure of the incident timeline prior to selecting the remediation playbook by the AI reasoning layer.

The paper emphasizes the significance of automation in minimizing manual intervention and offering recovery consistency. The creation of remediation playbooks could capture the operational expertise and make typical activities consistent and repeatable.The approach is complementary to the procedural one proposed in Flow-of-Action, where structured processes restrict the investigation of LLMs. Also, the approach is complementary to the multi-agent systems in AIOpsLab, LATS-RCA, and GALA where artificial intelligence is used for performing complex diagnostics. Thus, all these studies point to layered architecture, where AI performs reasoning and decision-making and deterministic operational processes control the execution of the actions. The relationship with RCA research is particularly important for RecoverIT.

Health-probe-based self-healing can answer “is the service unhealthy?” and playbooks can answer “what known recovery action should be attempted?”, but reliable autonomous incident handling also requires answering “why did the service become unhealthy?” RCAEval, MicroRCA-Agent, GALA, and other supplied papers address this diagnostic problem through telemetry correlation, multimodal reasoning, graphs, and LLM agents. RecoverIT combines these capabilities by placing evidence-driven

RCA before playbook selection, so that remediation is tied to a specific root-cause hypothesis rather than being triggered only by a generic health failure. The paper also supports the idea of using different remediation levels. Low-risk actions can be executed automatically, while actions with greater potential impact can require approval. This is important for RecoverIT's Safety Layer.

For example, restarting an unhealthy pod may be considered a low-risk automated action, while rolling back a production deployment or changing infrastructure configuration may require stronger validation or human approval. This policy-driven execution model reduces the risk of an LLM making an unsafe operational decision. For the Autonomous CI/CD Incident Triager & Self-Healer, the paper is particularly relevant to the Recovery Planner, Safe Action Executor, and Post-Remediation Verification modules. The design could adopt the concept of remediation playbooks and map each approved root-cause category to a set of permitted recovery actions.

2.16 CI/CD FAILURE REMEDIATION

This paper presents LogSage, an end-to-end Large Language Model-based framework designed specifically for detecting, diagnosing, and remediating failures in Continuous Integration and Continuous Deployment pipelines. The work is highly relevant to RecoverIT because it directly addresses the problem of automated failure investigation within CI/CD environments. Unlike general-purpose incident management approaches, LogSage focuses on the unique characteristics of CI/CD pipeline logs, which are often lengthy, heterogeneous, noisy, and difficult to analyze manually. The proposed LogSage framework follows a two-stage architecture. The first stage performs root cause analysis on failed CI/CD pipeline logs.

Instead of directly providing entire raw logs to an LLM, LogSage introduces a specialized preprocessing pipeline that identifies important error information and removes irrelevant log content. This decreases the load on the language model and makes it possible for the reasoning module to consider only high-quality pieces of evidence. The outputs of this step are a diagnosis and its explanation. The second step is focused on remedy generation. LogSage uses Retrieval-Augmented Generation to retrieve relevant historical resolution strategies from available knowledge sources.

The retrieved information is combined with the RCA result so that the system can generate an appropriate solution for the detected failure. Tool-calling is then used to translate the generated solution into an actionable remediation procedure. This is particularly important for RecoverIT because it demonstrates that an LLM-based system can progress beyond simply explaining an incident and can instead generate actions intended to resolve it. A major contribution of this work is its evaluation in both a public dataset and a large-scale industrial environment. The RCA stage achieved approximately 98% precision with near-perfect recall, demonstrating that specialized preprocessing can substantially improve LLM-based analysis of CI/CD logs.

The framework was also deployed in an industrial CI/CD environment where it processed more than 1.07 million executions over a year, demonstrating that LLM-based CI/CD failure management can operate at significant scale. The reported end-to-end precision exceeded 88%, showing the practical potential of combining automated diagnosis with solution generation. For RecoverIT, this paper offers a great basis for implementing a CI/CD failure analysis module, whereby the same technique of pre-processing and filtering of logs can be adopted. Rather than considering the whole CI/CD log as a context, it is possible to identify significant errors, pinpoint specific stages of the pipeline, create a timeline of the incident, and consolidate the findings with other information on deployment and source code. However, a future RecoverIT version could extend the scope of LogSage beyond CI/CD pipeline logs.

LogSage primarily focuses on failures within the CI/CD environment, whereas the proposed system is intended to connect software delivery events with production incidents. For example, the proposed agent can investigate whether a failed deployment, recent Git commit, configuration modification, or dependency update is responsible for a production failure. It can correlate CI/CD logs with application logs, infrastructure metrics, deployment history, and service health information. Another important extension is hypothesis-based root cause analysis. Rather than immediately accepting the first diagnosis generated by the LLM, the proposed system can generate multiple candidate root causes and evaluate each using evidence collected from different sources.

For example, after detecting a database connection failure, the agent could consider database outage, incorrect configuration, network failure, or a faulty deployment as separate hypotheses. It could then collect additional evidence to determine which

explanation is most strongly supported. The paper also provides an important foundation for the proposed Recovery Planner. LogSage demonstrates how retrieved historical knowledge and tool calling can be used to produce actionable remediation. The proposed system can extend this idea by introducing a dedicated safety layer between the LLM and infrastructure.

Actions such as deployment rollback, service restart, pipeline retry, and controlled scaling can be placed on an allowlist, while potentially destructive operations can require human approval. This approach can also be expanded through the introduction of a dedicated safety layer between the LLM and the infrastructure. Actions like rolling back the deployment, restarting the service, retrying the pipeline, and scaling can be whitelisted, whereas potentially destructive actions might need human approval. The proposed system can also expand on the research by adding closed-loop verification. Creating an action for remediation does not mean the problem has been solved yet. The agent needs to check error rates, service health, logs, and other relevant metrics after rolling back or restarting the application. If the system starts functioning again, the incident can be considered solved.

However, if the failure still exists, the agent should proceed to investigate again and refine its hypotheses. In summary, the main research opportunity offered by this paper is to expand the CI/CD failure diagnosis into a more general evidence-based incident response process. RecoverIT leverages CI/CD logs and the history of resolutions along with Git commits, deployment events, production telemetry, hypothesis-driven reasoning, remediation and testing of hypotheses, and post-remediation validation. In essence, it is a more general investigation and recovery process compared to the CI/CD failure diagnosis one.

This paper presents LogSage, an end-to-end Large Language Model-based framework designed specifically for detecting, diagnosing, and remediating failures in Continuous Integration and Continuous Deployment pipelines. The work is highly relevant to RecoverIT because it directly addresses the problem of automated failure investigation within CI/CD environments. Unlike general-purpose incident management approaches, LogSage focuses on the unique characteristics of CI/CD pipeline logs, which are often lengthy, heterogeneous, noisy, and difficult to analyze manually. The proposed LogSage framework follows a two-stage architecture. The first stage performs root cause analysis on failed CI/CD pipeline logs.

Instead of directly providing entire raw logs to an LLM, LogSage introduces a specialized preprocessing pipeline that identifies important error information and removes irrelevant log content. This decreases the load on the language model and makes it possible for the reasoning module to consider only high-quality pieces of evidence. The outputs of this step are a diagnosis and its explanation. The second step is focused on remedy generation. LogSage uses Retrieval-Augmented Generation to retrieve relevant historical resolution strategies from available knowledge sources.

The retrieved information is combined with the RCA result so that the system can generate an appropriate solution for the detected failure. Tool-calling is then used to translate the generated solution into an actionable remediation procedure. This is particularly important for RecoverIT because it demonstrates that an LLM-based system can progress beyond simply explaining an incident and can instead generate actions intended to resolve it. A major contribution of this work is its evaluation in both a public dataset and a large-scale industrial environment. The RCA stage achieved approximately 98% precision with near-perfect recall, demonstrating that specialized preprocessing can substantially improve LLM-based analysis of CI/CD logs.

The framework was also deployed in an industrial CI/CD environment where it processed more than 1.07 million executions over a year, demonstrating that LLM-based CI/CD failure management can operate at significant scale. The reported end-to-end precision exceeded 88%, showing the practical potential of combining automated diagnosis with solution generation. For RecoverIT, this paper offers a great basis for implementing a CI/CD failure analysis module, whereby the same technique of pre-processing and filtering of logs can be adopted. Rather than considering the whole CI/CD log as a context, it is possible to identify significant errors, pinpoint specific stages of the pipeline, create a timeline of the incident, and consolidate the findings with other information on deployment and source code. However, a future RecoverIT version could extend the scope of LogSage beyond CI/CD pipeline logs.

LogSage primarily focuses on failures within the CI/CD environment, whereas the proposed system is intended to connect software delivery events with production incidents. For example, the proposed agent can investigate whether a failed deployment, recent Git commit, configuration modification, or dependency update is responsible for a production failure. It can correlate CI/CD logs with application logs, infrastructure metrics, deployment history, and service health information. Another

important extension is hypothesis-based root cause analysis. Rather than immediately accepting the first diagnosis generated by the LLM, the proposed system can generate multiple candidate root causes and evaluate each using evidence collected from different sources.

For example, after detecting a database connection failure, the agent could consider database outage, incorrect configuration, network failure, or a faulty deployment as separate hypotheses. It could then collect additional evidence to determine which explanation is most strongly supported. The paper also provides an important foundation for the proposed Recovery Planner. LogSage demonstrates how retrieved historical knowledge and tool calling can be used to produce actionable remediation. The proposed system can extend this idea by introducing a dedicated safety layer between the LLM and infrastructure.

Actions such as deployment rollback, service restart, pipeline retry, and controlled scaling can be placed on an allowlist, while potentially destructive operations can require human approval. This approach can also be expanded through the introduction of a dedicated safety layer between the LLM and the infrastructure. Actions like rolling back the deployment, restarting the service, retrying the pipeline, and scaling can be whitelisted, whereas potentially destructive actions might need human approval. The proposed system can also expand on the research by adding closed-loop verification. Creating an action for remediation does not mean the problem has been solved yet. The agent needs to check error rates, service health, logs, and other relevant metrics after rolling back or restarting the application. If the system starts functioning again, the incident can be considered solved.

However, if the failure still exists, the agent should proceed to investigate again and refine its hypotheses. In summary, the main research opportunity offered by this paper is to expand the CI/CD failure diagnosis into a more general evidence-based incident response process. RecoverIT leverages CI/CD logs and the history of resolutions along with Git commits, deployment events, production telemetry, hypothesis-driven reasoning, remediation and testing of hypotheses, and post-remediation validation. In essence, it is a more general investigation and recovery process compared to the CI/CD failure diagnosis one.

2.17 CLOUD OBSERVABILITY RCA

The current paper proposes a solution that combines Large Language Models with cloud-native observability platforms to enable automatic root cause analysis and remediation in the cloud environment. This solution is highly relevant to the RecoverIT work because it covers the whole pipeline of the process starting with telemetry collecting and anomaly detecting up to root cause finding and automatic recovering. There is one main problem identified in the modern cloud-native solutions: applications are becoming more complex and consist of many microservices, containers, and other distributed components. Therefore, there is a lot of operational data generated in the form of metrics, logs, and traces. Rule-based monitoring tools may detect known conditions but cannot understand the complicated connections between signals.

Manually, people need to correlate these different signals to understand the real cause of an incident. To solve this problem, the proposed architecture is designed on multiple layers. The first layer is the layer of telemetry collecting using OpenTelemetry. Collecting metrics, logs, and traces for cloud-native applications takes place on this layer. Metrics contain information about performance of systems and applications, logs contain information about events, and traces contain information about requests flow.

Combining these sources allows the system to obtain a more complete representation of an incident. The next layer performs feature extraction and anomaly detection. Traditional machine learning techniques are used to process the collected telemetry before it is provided to the LLM. The framework uses techniques such as time-series anomaly detection, semantic analysis of logs, and service-dependency information. This design is important because it prevents the LLM from having to process the entire raw observability dataset.

Instead, potentially relevant anomalies and evidence are identified first and passed to the reasoning layer. The central component of the framework is an LLM-enhanced root cause analysis layer. A domain-adapted LLM is used to reason over the collected operational information and generate root-cause hypotheses. The system is designed to provide explanations for its diagnosis rather than simply returning a fault label. This aligns closely with RecoverIT requirement that every root-cause prediction should be supported by evidence such as logs, metrics, deployment events, or source-code changes.

The paper also introduces a domain adaptation strategy for the LLM. Operational data may include specific jargon, system behavior, and failures that are underrepresented in language model training data. That is why the proposed framework utilizes the operational data in order to fine-tune the model and to increase its reasoning capabilities concerning cloud-native incidents. Thus, the importance of domain-specific knowledge in application of LLMs to AIOps and incident management becomes apparent. Yet another important element of the framework is the automated remediation layer.

After the root cause of the problem has been recognized by the LLM, a remediation plan is generated and translated into the operational actions. It is important to note that the architecture includes safety measures in regards to remediation of problems, not arbitrary executing of actions generated by the model. Such an approach is highly relevant to the RecoverIT project, because infrastructure actions have to be limited in some way or another. At the same time, the framework has an important feedback loop between the remediation and system monitoring layers.

This is an important characteristic of a self-healing system because simply executing a recovery command does not guarantee that the original incident has been resolved. The system must verify whether the relevant symptoms have disappeared. Experimental evaluation reported substantial improvements in incident-management performance. The proposed framework achieved approximately 95% F1-score for anomaly detection, an 84.2% reduction in mean time to resolution compared with rule-based approaches, and an automation rate of approximately 91% for common incidents. These results demonstrate the potential of combining traditional observability techniques with LLM-based reasoning and automated remediation.

For RecoverIT, this paper provides the foundation for the production observability side of the architecture. Just like the proposed system can do, it is possible for the system to gather metrics, logs, and traces to be used as evidence during incident investigations. Nevertheless, the difference between the proposed system and a typical cloud-native self-healing system is that, unlike the latter, the former does not just gather runtime operational signals. It also takes into consideration the signals related to the software delivery process including Git commits, CI/CD execution results, deployment versions, configuration changes, and pull requests.

For example, if a service begins producing errors immediately after a deployment, the agent should not only inspect runtime telemetry. It should also investigate what changed in the new release, compare it with the previous healthy version, examine CI/CD results, and determine whether the deployment is a plausible cause. The paper also supports RecoverIT's use of multimodal evidence. Logs, metrics, and traces provide complementary views of the same incident. a future RecoverIT version could extend this multimodal approach by introducing a fourth category of evidence: software-delivery context. Git history, deployment records, pipeline results, configuration changes, and release information can be correlated with runtime telemetry to build a complete incident timeline.

A further extension is the introduction of explicit risk-aware remediation. In this case, the suggested system will have to categorize the recovery action based on its possible effect. The lower-risk actions such as restarting the service or retrying the pipeline will have to be automatically performed using preset boundaries, whereas high-risk actions like database change or infrastructure change that could damage the system will have to be manually approved.

2.18 COMPARISON AND RESEARCH GAP

Research strand	Contribution	Remaining gap for this project
LLM agents	Iterative tool use and evidence retrieval	Need bounded cost and structured provenance
Log-focused RCA	Filters high-value log evidence	Need to join software delivery with runtime context
Microservice RCA	Causal and multimodal reasoning	Need source identity, timeline and abstention
CI/CD failure analysis	Pipeline-specific diagnosis	Need production incident correlation
Self-healing systems	Recovery procedures and monitoring	Need approval and execution safety before deployment

Table 2.1 Literature comparison

The common gap is the joining of delivery events and runtime observations into a single evidence trail while keeping conclusions and proposed actions accountable. RecoverIT addresses that gap at the investigation and recommendation layers. It does not claim to exceed published RCA accuracy or operational efficiency results, because a comparable labelled incident benchmark and production evaluation are not part of the verified current results.

CHAPTER 3

METHODOLOGY AND SYSTEM DESIGN

3.1 DESIGN PRINCIPLES

Observation comes before inference. Each observation retains an origin, identifier and time context so a hypothesis can cite a concrete record. The model does not become the authority for whether the cited record exists. This principle appears in typed source batches, normalized evidence, context construction and final policy checks.

The runtime is configured explicitly. A settings file chooses the reasoning provider, model, source adapters, concurrency, timeouts and checkpoint location. Automatic live composition requires a real provider; test doubles are only injected by tests. This avoids accidental production runs over canned scenarios.

Investigations are bounded. The graph controls collection rounds through an investigation budget and stopping logic. The completion is not necessarily automatic just because the LLM made an acceptable claim. The system can produce an inconclusive answer with unfinished criteria if there is not enough information for a proper diagnosis.

3.2 SYSTEM ARCHITECTURE

The application has three broad layers. The boundary layer loads a canonical incident and discovers configured source capabilities. The evidence layer collects, validates, normalizes, stores and combines records into a timeline and context snapshot. The reasoning layer assesses gaps, proposes queries and hypotheses, evaluates stopping criteria, ranks supported hypotheses and optionally prepares an advisory remediation plan. A runner projects graph state into a stable result for CLI and HTTP clients.

The LangGraph state is deliberately composed of serializable data rather than service instances. Dependency objects remain outside graph state and are injected into nodes. Checkpoints can therefore retain investigation progress without trying to serialize live HTTP clients, callbacks or repositories. A SQLite checkpointer is used for live runs; tests can use in-memory state.

The system’s source adapters are read-only. This boundary permits investigation across operational systems while restricting the runtime’s authority. Each adapter advertises

what it can query and returns a typed result with source status, records and warnings. Unavailable sources are explicit gaps rather than fabricated negative evidence.



Figure 3.1 RecoverIT investigation workflow

3.3 INCIDENT INTAKE AND CONTRACTS

A canonical IncidentSeed carries the incident identifier, affected service, environment, severity, detection and receipt times, and summary. Typed Pydantic contracts validate data crossing module boundaries. Schema versioning and canonical JSON reduce ambiguity between collection, evidence, timeline and reasoning modules.

The source capability catalogue describes what information each configured adapter can supply. A query plan asks for selected fields and time windows. The collection service dispatches supported queries with bounded concurrency and timeout handling, then reports successes and failures separately. This means a failed adapter can be discussed as missing coverage without being confused with a clean result.

The contract boundary is especially important for an AI-assisted system. The output of the free-form model is shaped according to the requirement. Identifiers, time fields, evidence references and status enums are validated early. Invalid data becomes an explicit failure or blocked recommendation rather than silently entering the final report.

3.4 SOURCE COLLECTION

Configured adapters provide read-only access to distinct source categories, including logs, metrics, Git changes, deployments, pipeline records and health signals where configured. The live runtime builds only the adapters specified in its trusted settings. Capabilities allow the query planner to adapt to a particular deployment rather than assume that every incident has all sources.

The collection design separates retrieval from interpretation. Adapters return original record IDs and timestamps; they do not decide a root cause. The collection service aggregates query results, retains any partial failure and prevents crashing of the analysis as one source times out. This allows later stages to reason about evidence coverage.

Source endpoints and credentials are present only in server-side configurations. Operational content is considered to be untrusted data and not the instruction to the graph. Before the context goes through to the model, the record is validated and redacted. This is important since logs and configuration payload might carry some secret or adversarial text.

3.5 EVIDENCE PROCESSING AND PROVENANCE

The evidence layer normalizes timestamps to a common UTC representation while preserving source event time and observation time. It assigns stable identities to records, classifies their type and derives a quality assessment. Original source IDs remain available for traceability. These steps support comparison across heterogeneous systems without erasing where the data came from.

Deduplication reduces repeated observations of the same underlying event. For example, a repeated log line should not create several independent votes for a hypothesis. The evidence repository records normalized items and allows deterministic filtering, ordering and pagination. This makes repeated investigations more reproducible.

Redaction is applied before summaries are made available to reasoning. Raw secrets and tokens should not be copied into a prompt or final recommendation. The incident context contains evidence summaries and source coverage so the hypothesis stage can explain what is supported and what remains missing.

3.6 TIMELINE AND CONTEXT CONSTRUCTION

Timeline building involves mapping normalized evidence to typed events which are sorted according to event times with deterministic tie breaking. Temporal relationships express sequence and association without automatically asserting causality. The difference here is significant since although prior occurrence of a deployment before an error may be valuable evidence, order alone doesn’t make a causation assertion.

Context construction combines the incident seed, source coverage, evidence summaries, timeline events and relationships into a snapshot. It is the information boundary consumed by the reasoning services. The design prevents those services from reaching back into raw source adapters arbitrarily after context publication.

Contextual information is needed to ensure that there is sufficient selectiveness but at the same time the amount of contextual information should not be too limited. This is because insufficient contextual information will lead to a hypothetical diagnosis, while too much information leads to lack of causal evidence. The graph therefore begins by assessing missing information and planning focused queries. It can update the context after a collection round and decide whether another round would be informative.

3.7 INVESTIGATION GRAPH AND STATE

The compiled graph contains initialization, gap assessment, query planning, evidence collection, context building, hypothesis generation, stopping evaluation, ranking and inconclusive completion nodes. When a remediation planning service is available, completed and inconclusive paths both reach a final plan node. An inconclusive path yields escalation and verification guidance without recommending a production change.

Conditional edges route after query planning and after stopping evaluation. The graph can revisit the collection while the budget remains, or finish with the best supported output. Persistent checkpoints are keyed by incident ID. The runner supports both a new run and resumption of a saved state.

Progress events are dispatched through a task-local callback binding rather than stored in graph state. This helps concurrent investigations avoid sending one incident's events to another client. The runner converts graph output into a stable result containing status, hypotheses, timeline, evidence, budget usage and remediation guidance.

3.8 HYPOTHESIS RANKING AND STOPPING

A candidate hypothesis names a suspected cause, affected component, supporting and contradicting evidence, and unresolved questions. The ranking engine calculates a reproducible score using positive evidentiary factors and penalties. Source independence guards against treating duplicates as corroboration. Symptom coverage checks whether a cause explains the reported failure; temporal consistency checks event order; change consistency considers direct delivery evidence.

Contradiction and lack of causation make confidence lower. High correlation in symptoms does not guarantee the cause, thus, the implementation distinguishes confidence in symptom and causation in its report. Ranking is useful only if the

evidence references remain inspectable, and final output records the score breakdown for that purpose.

Stopping rules consider budget limits, information sufficiency and the quality of available hypotheses. Abstention is a valid result. It prevents a presentation layer from turning a weak top-ranked hypothesis into a definitive root cause simply because a list must have a first item.

3.9 RECOMMENDATION-ONLY REMEDIATION

The remediation service requests a typed candidate plan from the configured reasoning provider. The plan may contain requirements, descriptive actions, anticipated results, verification tests, reversal instructions, and risk categorization. This service advises on the matter and the output is subject to human approval. There are no commands executed here.

A deterministic policy disallows diagnoses or evidence IDs which cannot be confirmed or contradict each other, executable shell commands and sensitive credentials. Failure of the provider to return a result in time will be regarded as a blocked fallback. Inconclusive reasoning results in a blocked plan with escalation recommendations.

The approach is intentionally narrower in scope than the initial idea of self-healing. It enables a safe interaction between services without any assumption that approval, execution or post-execution checks are already in place. Those features have to be provided independently and tested.

3.10 INTERFACES AND DEPLOYMENT

The command-line interface accepts a runtime configuration and canonical incident file and writes an investigation report. A FastAPI service exposes the same underlying runner to HTTP clients. The web interface displays status, progress, evidence and ranked results. These are presentation surfaces over one application result, not separate diagnosis engines.

Live settings specify the provider and source adapters; a missing or invalid configuration is rejected. Checkpoint storage can use an application data path, and runs should be scoped to incident identifiers. This makes the system suitable for local demonstration and controlled integration, subject to the availability and permissions of external sources.

The development setup requires Python 3.11 or later and typed dependencies including Pydantic, FastAPI, httpx, LangGraph and SQLite checkpoint support. Test dependencies are separate.

3.11 EVALUATION DESIGN

The current repository demonstrates functional correctness primarily through unit, contract and integration tests. A broader RCA evaluation would require labelled incidents with root-cause ground truth, source availability labels and consistent benchmark conditions. The measures need to capture root cause correctness, citation correctness, quality of abstention, elapsed investigation time and operational costs.

A fair comparison would run a manual or rule-based baseline on the same incidents and source snapshots. It would separately evaluate cases with missing sources, contradictory records and irrelevant log volume. Remediation recommendations should be graded for evidence grounding, safety, clarity and verification guidance before any execution path is considered.

No published comparative metric is claimed for RecoverIT in this report. The test suite establishes behaviour of implemented contracts and flows, but does not by itself establish production diagnostic accuracy or reduced mean time to resolution.

Module	Input	Output
Collection	Incident, source capabilities, query plan	Raw evidence batch
Evidence processing	Raw records	Normalized evidence and repository entries
Context	Evidence and incident	Timeline and context snapshot
Reasoning	Context and budget	Candidate hypotheses
Ranking	Candidates and evidence	Ranked hypotheses or inconclusive status
Remediation	Ranked output and context	Advisory plan or blocked guidance

Table 3.1 Module interfaces

Boundary	Control
External operational sources	Read-only registered adapters
Untrusted record content	Validation, normalization and redaction
LLM output	Typed contracts and deterministic ranking/policy
Production changes	No write-capable executor in current runtime

Table 3.2 Safety boundaries

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 REPOSITORY IMPLEMENTATION

The repository is organized around contracts, collection, evidence, timeline, investigation graph, reasoning, remediation and the RecoverIT application. The package metadata declares Python 3.11 or later and includes Pydantic, httpx, FastAPI, Uvicorn, LangGraph and SQLite checkpoint support. The README provides installation and CLI instructions. Source modules implement the live path while the docs directory records design decisions, interfaces and handoffs. Some older architecture material describes a larger proposed system; this chapter identifies what is present in current code.

4.2 RUNTIME COMPOSITION AND GRAPH

The `recoverit.composition` module validates a JSON runtime file and builds a graph with selected source adapters and a real reasoning provider. It rejects automatic assembly of test or benchmark modes. Live runs use an SQLite checkpoint saver unless another backend is injected, while tests may use an in-memory saver. The `investigation.graph.builder` module assembles ten named nodes and routes completed or inconclusive investigations to recommend planning when that service is configured.

The `recoverit.runner` module invokes the graph with a canonical incident and investigation budget. It exposes status, stop reason, ranked hypotheses, timeline events, evidence, provider identity, budget usage and optional remediation plan. Its Markdown report shows a factor-level score breakdown and labels remediation as a human-reviewed suggestion. A resume method loads checkpointed graph state for an incident ID.

4.3 COLLECTION AND EVIDENCE

The collectors package implements source-specific read-only adapters and a collection service. The evidence package validates inputs, normalizes timestamps and record identity, redacts sensitive content, deduplicates observations and stores results through repository contracts. Timeline services map evidence to events and relationships. Context publication presents a summarized, typed snapshot to reasoning.

The separation between raw collection and normalized context is visible in both package boundaries and tests. It prevents adapter-specific payloads from directly becoming conclusions. Failure information remains available as source status and warnings, so an unavailable source contributes to uncertainty rather than a false negative finding.

4.4 REASONING AND RANKING

The reasoning package contains a provider interface, live client, hypothesis services, ranking engine and stopping logic. Test-only fake and recorded providers exist to make graph behaviour reproducible; live composition does not select them implicitly. Ranking features quantify independent source support, symptom coverage, temporal consistency, direct change evidence, specificity and prediction support, then apply contradiction and missing-evidence penalties.

The final report keeps both the score and factor breakdown. This is useful for review because a reader can inspect why one cause ranked above another. The process is still based on the level of evidence and model-generated candidate hypotheses; therefore, a high score is not the true probability of being correct.

4.5 REMEDIATION POLICY

The current remediation module produces recommendation-only plans. It asks the reasoning provider for a candidate plan, then validates the plan against deterministic policy. The validator blocks plans on low confidence, contradictions, unknown evidence IDs, executable command syntax or sensitive content. Provider failure and inconclusive investigations yield blocked plans and escalation guidance.

The implemented plan describes steps for a human operator and may include verification and reversal guidance. It does not open approval tickets, invoke infrastructure tools, execute commands or measure post-action recovery. This distinction is material to the conclusion of the project.

4.6 INTERFACES

The CLI runs a configured investigation from incident JSON and can write a Markdown report. The HTTP service and web client display the same application result and progress updates. The interface should be interpreted as a view of the investigation; the evidence record and typed graph output remain the authoritative technical artefacts.

The report generated by the runner contains ranked hypotheses, evidence scores and factor breakdowns, unresolved criteria when present, and advisory remediation information. This allows a reviewer to compare output with the source evidence and determine whether the recommendation is justified.

4.7 TESTING AND VALIDATION

To check the live investigation path, a separate case study was performed with HarborShop, a locally hosted checkout application. The first revision stored its product and order data in a provisioned SQLite directory. A later one-line configuration change pointed the service to a different storage directory that had not been created. This gave the test a known fault which could be compared with the cause reported by RecoverIT.

The same traffic script was used on both revisions. In the healthy run, 15 valid checkout requests returned HTTP 201 and the readiness endpoint returned HTTP 200. After the change, all 15 valid checkout requests returned HTTP 500 and readiness returned HTTP 503. The liveness endpoint continued to return HTTP 200, while deliberately invalid checkout inputs still returned HTTP 400. Application and database logs recorded the failed requests and the database file opening error. These observations establish the test conditions independently of the investigation result.

RecoverIT was then run with local Git changes, versioned configuration, structured log files, the HTTP readiness check and request metrics enabled as evidence sources. The live run completed in 16.45 seconds. Figure 4.1 shows the investigation trace and the incident summary recorded by the interface.



Figure 4.1 Live investigation trace and incident summary

The order storage path change was ranked first as a configuration regression, with an evidence score of 76.45 out of 100. An infrastructure failure was ranked second at 49.70. The leading explanation agreed with the known one-line change and with the failed readiness checks. Figure 4.2 records the ranked output and the cited evidence.

The percentages in this screen are internal scoring values; they are not measured probabilities of diagnostic correctness. In particular, the low symptom-confidence value depends on how individual log records are counted in this case.

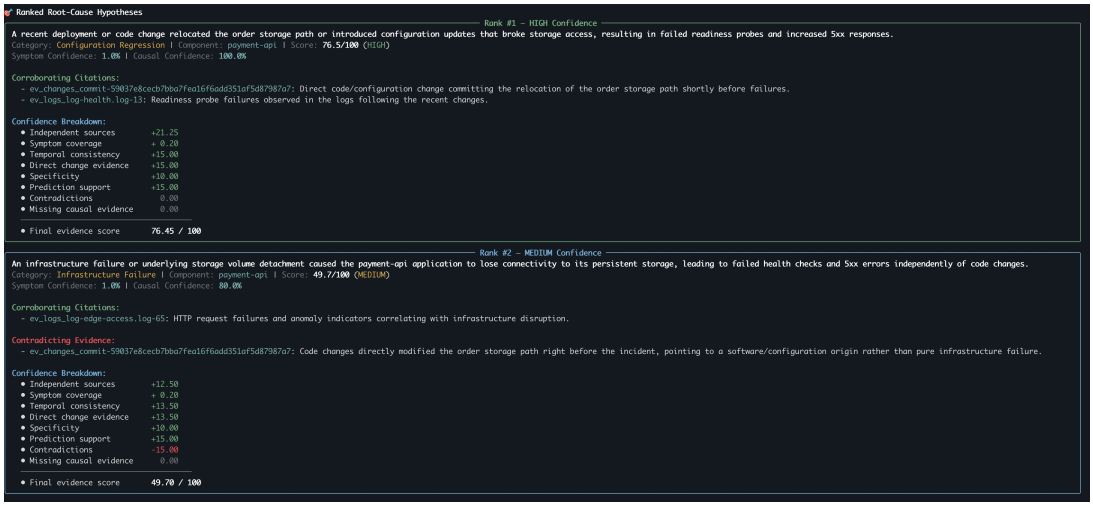


Figure 4.2 Ranked root-cause hypotheses for the controlled incident

The system also produced a proposed sequence for reviewing the configuration, checking the readiness and application logs, correcting the path, and verifying recovery. Figures 4.3 and 4.4 show this output. These are recommendations requiring human review. No change was executed by RecoverIT and no post-remediation recovery was measured in this run. Some references to centralized dashboards and a deployment pipeline are general suggestions; the case study used local logs and a local service instead.

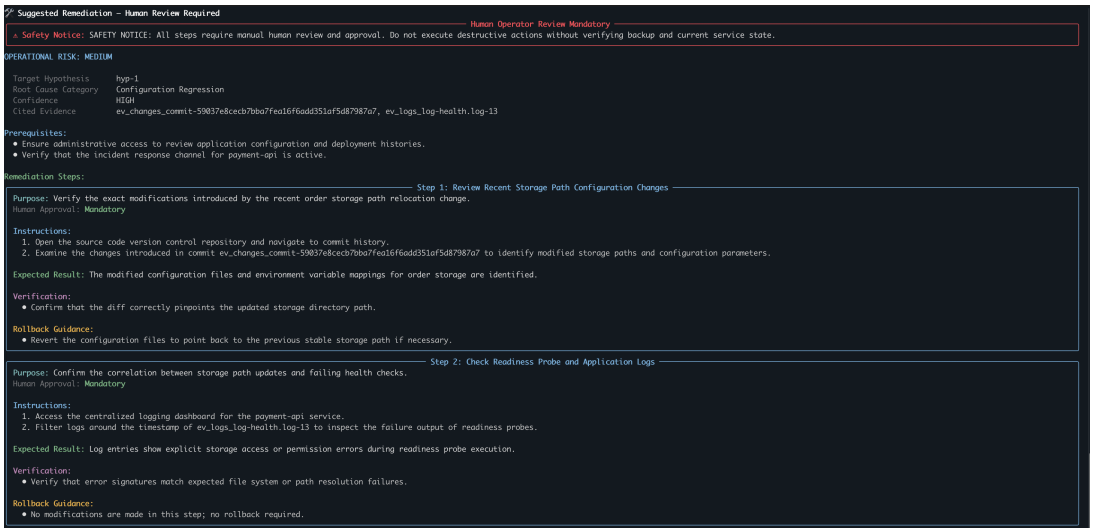


Figure 4.3 Advisory remediation output with review and log inspection steps

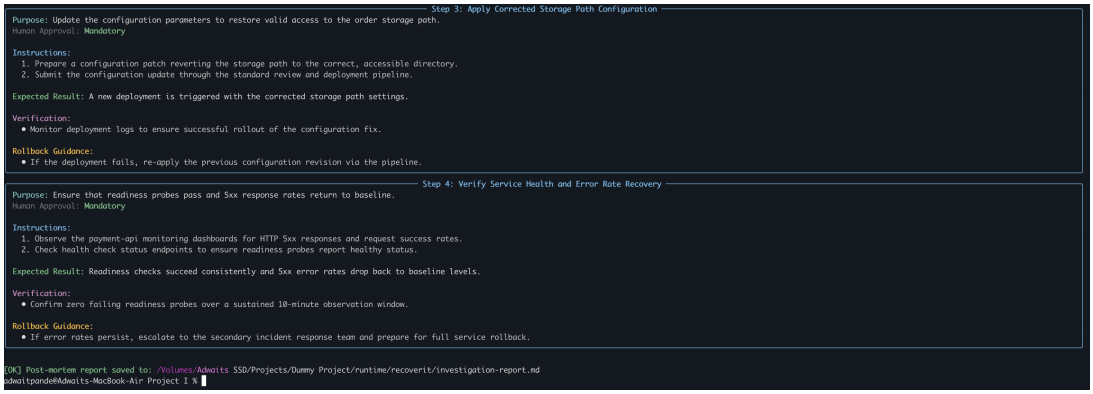


Figure 4.4 Advisory configuration and recovery verification steps

This exercise shows that the implemented workflow identified the planted fault in one controlled incident using live evidence from several sources. It does not establish accuracy across unfamiliar incidents or any reduction in resolution time. The case study therefore extends the repository test result with an end-to-end example, while the broader evaluation requirements remain open.

4.8 RESULTS ANALYSIS AND LIMITATIONS

The observed outcome is an integrated, test-verified investigation workflow with typed boundaries and safe advisory remediation. The tests support the claim that the implemented modules behave as specified by their assertions. They do not show how accurately RecoverIT diagnoses unfamiliar production incidents. There were no comparative experiments conducted between the software and a human or any other tools in this case study, nor was there any reduction in the mean time to resolution measured.

The solution is constrained by lack of full source code coverage, inaccurate telemetry, behavior of the providers, and the plausibility of alternative hypothesis. Ranking based on deterministic algorithms mitigates some potential sources of failure, but it does not take the place of labeled calibration. The policy module prevents some unsafe recommendations, but a thorough security analysis and approval process is required before actual execution.

The original self-healing title describes a long-term direction. The current implementation ends at a human-reviewed recommendation. It has no execution engine or closed-loop confirmation that a service recovered. This gap is documented explicitly so that readers can distinguish implemented work from the broader architecture proposal and literature examples.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

RecoverIT demonstrates a bounded approach to CI/CD incident investigation. It brings configured read-only source records into typed evidence, constructs a timeline and incident context, generates and ranks root-cause hypotheses, and reports the evidentiary factors behind its ordering. The LangGraph workflow supports explicit state transitions and checkpointing. The result is designed for operator review rather than automatic acceptance of a model response.

The current implementation also prepares remediation guidance under deterministic safety rules. It blocks change recommendations when confidence, evidence grounding or content safety checks fail. This is the principal safety conclusion of Project I: an investigation assistant can provide structured suggestions while retaining an explicit human decision boundary. The repository test run establishes a passing software verification baseline, subject to the limits of test coverage.

The project has not yet demonstrated production RCA accuracy, faster resolution, safe execution of infrastructure actions or verified self-healing. Those outcomes remain research and engineering goals. Presenting them as achieved would exceed the available evidence.

5.2 FUTURE WORK

First comes the labelled incident evaluation. Incidents must involve root causes that are known, diverse sources, missing data, and conflicting observations. The evaluation should capture top-one and top-k diagnostic accuracy, evidence-reference accuracy, confidence calibration, abstention quality, time spent, and cost to the provider. The rule-based or manual baseline should be run on the same immutable source snapshots.

Future work should focus on expanding the set of sources and enhancing causality inference. More adapters, dependencies, and links to deployment changes may help provide better context. Controlled experiments should confirm that each new source helps diagnose rather than simply adding to the length of the prompt. Adversarial tests should focus on prompt injection into logs, secret leaks, and deceptive event ordering.

Future recovery functionality would call for a separate list of approved actions, identification and authorization, human confirmation for changes, bounded execution, safety in case of rollback, and checking after an action is done. It is only after all of those safeguards are in place that a system can claim its self-healing capability.

REFERENCES

[1] Manohar, N. V., Shukla, R., & Abi, S. (2025, December). Autonomous Multi-Agent System for Integrated SRE and Self-Healing in Cloud-Native Environments. In International Conference on Smart Systems and Social Management (ICSSSM 2025) (pp. 525-535). Atlantis Press.

[2] Roy, D., Zhang, X., Bhave, R., Bansal, C., Las-Casas, P., Fonseca, R., & Rajmohan, S. (2024, July). Exploring llm-based agents for root cause analysis. In Companion proceedings of the 32nd ACM international conference on the foundations of software engineering (pp. 208-219).

[3] Wittkopp, T., Wiesner, P., & Kao, O. (2024, August). Logrca: Log-based root cause analysis for distributed services. In European Conference on Parallel Processing (pp. 362-376). Cham: Springer Nature Switzerland

[4] Wang, T., & Qi, G. (2024). A comprehensive survey on root cause analysis in (micro) services: Methodologies, challenges, and trends. arXiv preprint arXiv:2408.00803.

[5] Pham, L., Ha, H., & Zhang, H. (2024, October). Root cause analysis for microservice system based on causal inference: How far are we?. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (pp. 706-715).

[6] Pham, L., Zhang, H., Ha, H., Salim, F., & Zhang, X. (2025, May). Rcaeval: A benchmark for root cause analysis of microservice systems with telemetry data. In Companion Proceedings of the ACM on Web Conference 2025 (pp. 777-780).

[7] Chen, Y., Shetty, M., Somashekar, G., Ma, M., Simmhan, Y., Mace, J., ... & Rajmohan, S. (2025). Aiopslab: A holistic framework to evaluate ai agents for enabling autonomous clouds. Proceedings of Machine Learning and Systems, 7.

[8] Pei, C., Wang, Z., Liu, F., Li, Z., Liu, Y., He, X., ... & Pei, D. (2025, May). Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis. In Companion Proceedings of the ACM on Web Conference 2025 (pp. 422-431).

[9] Ji, C., & Luo, H. (2025, May). Cloud-based ai systems: Leveraging large language models for intelligent fault detection and autonomous self-healing. In 2025 IEEE 7th International Conference on Communications, Information System and Computer Engineering (CISCE) (pp. 226-229). IEEE.

[10] Tang, P., Tang, S., Pu, H., Miao, Z., & Wang, Z. (2025). MicroRCA-Agent: Microservice root cause analysis method based on large language model agents. arXiv preprint arXiv:2509.15635.

[11] Naakka, A., Wang, Y., & Mäntylä, M. V. (2026, August). LATS-RCA: Language Agent Tree Search for Root Cause Analysis in Microservices. In Euromicro Conference on Software Engineering and Advanced Applications (pp. 196-212). Cham: Springer Nature Switzerland.

[12] Tian, Y., Liu, Y., Chong, Z., Huang, Z., Li, Y., & Jacobsen, H. A. (2026, October). GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in

Microservices. In Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (pp. 55-67).

[13] Mhatre, A., & Shaikh, A. (2025). LLM for Automated Root Cause Analysis in Microservices Architectures. Journal of Information Systems Engineering and Management, 10(13), 449-460.

[14] Kalakoti, M. K. R., & Kalakoti, M. K. R. (2025). AI-augmented self-healing infrastructure: Combining health probes with remediation playbooks. Journal of Information Systems Engineering and Management, 10(58s), 1147-1157.

[15] Xu, W., Luo, J., Huang, T., Sui, K., Geng, J., Ma, Q., ... & Cai, P. (2025, November). LogSage: An LLM-based framework for CI/CD failure detection and remediation with industrial validation. In 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE) (pp. 3742-3753). IEEE.

[16] Wang, C., Yuan, T., Hua, C., Chang, L., Yang, X., & Qiu, Z. (2025, November). Integrating large language models with cloud-native observability for automated root cause analysis and remediation. In Proceedings of the 2025 3rd International Conference on Artificial Intelligence, Systems and Network Security (pp. 327-334).

APPENDIX 1 RUNTIME INTERFACES

The live command accepts a trusted runtime configuration, a canonical incident JSON file and an optional report destination. The runtime file chooses the reasoning engine, model, checkpoint storage, data adapter, concurrency, and timeout for queries. The incident file holds the incident id, involved service, environment, level, timestamps and description. A good run outputs status, ordered hypotheses, evidence and timeline entries, budget consumption, and optional recommendation for remediation.

Usage Example: python -m recoverit.cli investigate --config path/to/runtime.json --incident path/to/incident.json --report path/to/investigation-report.md

The sample files in `docs/` describe the required formats. The files need to be checked and adjusted to contain a valid endpoint and credentials before being used for a real run. Fake test providers and benchmark fixtures are not part of the live execution flow.

APPENDIX 2 CONTRACTS AND SOURCES

Contract or source	Role
IncidentSeed	Canonical incident identity and time context
SourceCapabilityCatalog	Advertised query support and availability
RawEvidenceBatch	Typed source retrieval result
EvidenceRecord	Normalized, provenance-bearing observation
IncidentContextSnapshot	Published context for reasoning
RankedHypothesisSet	Ordered diagnosis and evidence links
RemediationPlan	Human-reviewed guidance or blocked escalation

Table A.1 Source capability map

The contract sequence ensures that a reader can follow an observation from its origin through normalization, timeline construction, hypothesis support and final recommendation. The provenance and evidence-ID checks are particularly important when a model-generated statement cites operational data.

APPENDIX 3 TEST MATRIX

INPUT AND CONTRACTS

Malformed incidents, missing required fields and invalid enums should be rejected before collection.

COLLECTION FAILURES

An unavailable or timed-out adapter should produce a partial result with warnings rather than a fabricated clean reading.

EVIDENCE QUALITY

Timestamps, redaction, deterministic identity and repeated observations should be handled consistently.

GRAPH ROUTING

The graph should stop on budget exhaustion and return an inconclusive result when evidence is inadequate.

RANKING

Contradictory and missing causal evidence should reduce a candidate's score.

REMEDIATION

Unknown evidence IDs, shell-like syntax and low confidence should produce a blocked recommendation.

PRESENTATION

CLI, API and web views should preserve the same investigation status and provenance links.

APPENDIX 4 INDIVIDUAL CONTRIBUTION

Candidate name: _____

Register number: _____

Project role and modules implemented:

_____

Design, implementation, testing and documentation contributions should be filled by the candidate and verified by the guide before submission. The report describes team project work, while this statement records the individual contribution required by BCSE497J.